Database Systems

Piya Limcharoen

Version 2026/1

Contents

Chapter 1: Transaction Management and Concurrency Control 5
1.1 Chapter Overview . . . 5
1.2 Data Abstraction and Data Independence . . . 9
1.3 Data Abstraction . . . 11
1.3.1 Three Levels of Abstraction . . . 11
1.4 Data Independence . . . 14
1.5 Basic Database Operations (CRUD) . . . 18
1.6 Database Transactions . . . 20
1.7 ACID Properties . . . 25
1.7.1 Atomicity (All or Nothing) . . . 31
1.7.2 Consistency (Correctness) . . . 38
1.7.3 Isolation (Independence) . . . 47
1.7.4 Durability (Persistence) . . . 60

1.8 Popular Database Systems . . . 66
1.8.1 DBMS (Database Management System): . . . 66
1.8.2 DBMS vs RDBMS: . . . 66
1.8.3 Popular Database Systems: . . . 67
1.8.4 Oracle Database . . . 67
1.8.5 MySQL / MariaDB . . . 68
1.8.6 PostgreSQL . . . 68
1.8.7 Microsoft SQL Server . . . 68
1.8.8 SQLite . . . 69
1.8.9 MongoDB . . . 69
1.8.10 Vector Databases . . . 69
1.8.11 ACID Support Status . . . 71
1.9 Chapter Summary . . . 72
1.10 Activity/Homework: Banking System GUI . . . 74
1.10.1 Tools . . . 74
1.10.2 Database Schema . . . 75
1.10.3 Lab Tasks . . . 76

Copyright Notice

Copyright © 2026 Piya Limcharoen. All rights reserved.

This lecture note was authored by Piya Limcharoen. Artificial Intelligence (AI) was utilized as a tool for language enhancement and grammatical correction; however, all intellectual property, original concepts, and the structure of this work belong exclusively to the author.

Terms of Use

This document is provided for educational purposes only and is intended for the exclusive use of students currently enrolled in this course.

The reproduction, distribution, public display, or transmission of this material, in whole or in part, in any form or by any means—electronic, mechanical, photocopying, recording, or otherwise—is strictly prohibited without the prior written permission of the author. Students may not share, upload, or distribute these notes outside of the official class environment.

Warning

Any unauthorized use of this material will be considered academic misconduct and will be subject to university disciplinary procedures. Furthermore, such an act may constitute an infringement of copyright law and is subject to legal action.

Chapter 1:
Transaction Management and Concurrency Control

Chapter 1: Transaction Management and Concurrency Control

1.1 Chapter Overview

This chapter explores the fundamental concepts of Database Management Systems (DBMS) and transactions, focusing on:

- **Data Abstraction & Independence:** How databases hide complex physical storage details through a 3-schema architecture (External, Conceptual, Internal).
- **ACID properties** (Atomicity, Consistency, Isolation, Durability) that guarantee data integrity during transactions.
- How modern systems like MySQL and PostgreSQL manage concurrent user access using protocols like **Two-Phase Locking (2PL)** and **Multi-Version Concurrency Control (MVCC)**.
- How databases survive failures using **Write-Ahead Logging (WAL)** and recovery mechanisms.
- **The DBMS Landscape:** The difference between DBMS and RDBMS, and the various database categories (Relational, Document, Key-Value, Vector).

Core Questions We Will Answer:

- **Abstraction** → How do databases separate what users see from how data is stored?
- **ACID** → What guarantees do we want from our database?
- **Atomicity & Durability** → How do we handle crashes and errors? (Logging & WAL)
- **Isolation** → How do we allow concurrent transactions without chaos? (2PL & MVCC)
- **Real World** → Which databases (Oracle, MySQL, SQLite, MongoDB) are used for what?

Learning Objectives

By the end of this chapter, you will be able to:

- **Data Abstraction:** Hiding physical storage details from users.
  - **Three levels:** Physical (Internal), Logical (Conceptual), and View (External) schemas.
  - **Data independence:** Ability to modify one level without affecting others.
- **Transactions:** Grouping operations into a reliable unit.
  - **Definition:** A single logical unit of work in a database.
  - **Data integrity:** Ensuring data stays valid despite crashes or concurrency.
- **ACID Properties:** The fundamental guarantees for transaction safety.
  - **Atomicity:** All or nothing operations.
  - **Consistency:** Transforming data between valid states.
  - **Isolation:** Safely managing concurrent transactions.
  - **Durability:** Ensuring committed changes are permanent.
- **Concurrency Control:** Safely managing simultaneous user access.
  - **Anomalies:** Issues like dirty reads and lost updates.
  - **Isolation levels:** Trade-offs between correctness and performance.
- **Database Systems:** Comparing real-world implementations.
  - **Transaction handling:** Approaches used in MySQL, PostgreSQL, etc.

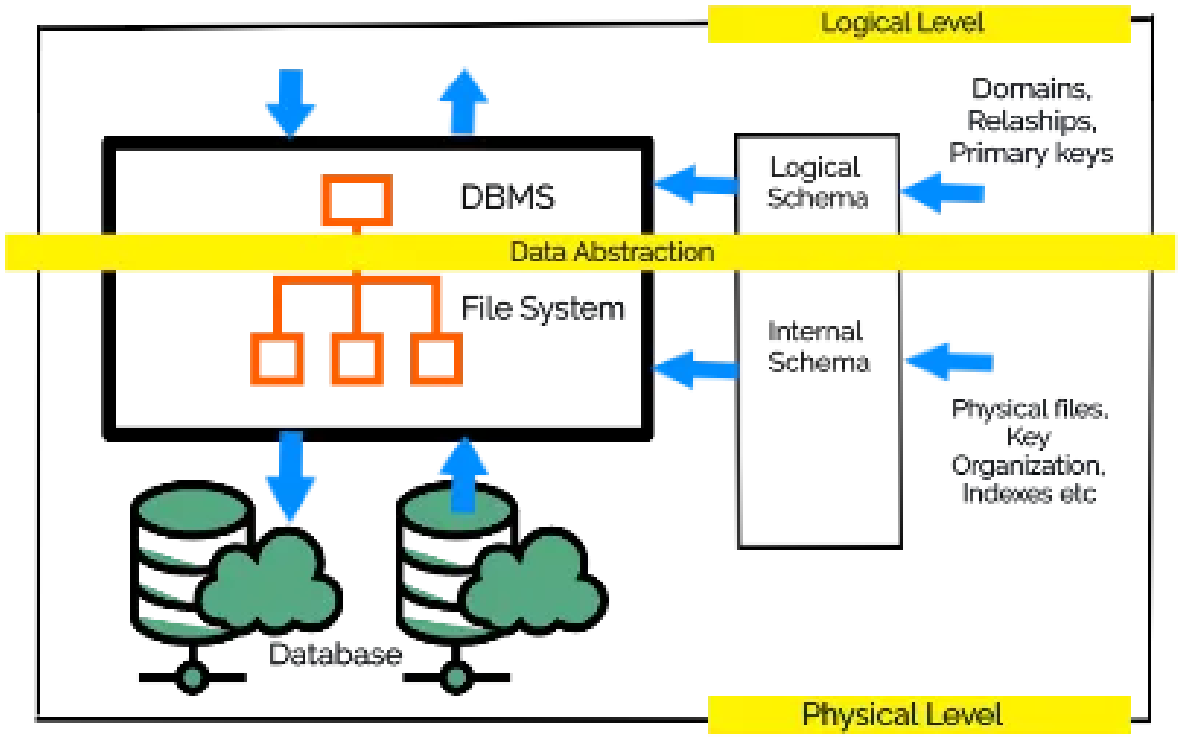


Figure 1: Data Abstraction and Data Independence

Source: ExamRadar

1.2 Data Abstraction and Data Independence

1. **Data Abstraction:**
   Hides unnecessary database details by providing users with a simpler, higher-level view of the data.
2. **Data Independence:**
   Allows changes at one level of the database architecture without requiring changes at the higher level.

Real World Context: The Car Analogy

Think of a car.

1. **External Level (Driver):** You interact with the steering wheel, pedals, dashboard, and other controls. You do not need to know how the engine or other internal components work in order to drive the car.
2. **Internal Level (Mechanic):** A mechanic works with the components "under the hood," such as the engine, fuel injection system, pistons, and spark plugs. These details are hidden from the driver.

Data Abstraction: Allows you to be the driver without being the mechanic.
Data Independence: Allows the mechanic to change what is under the hood without requiring the driver to change how they drive.

Concept	**Levels / Types**
Data Abstraction	External, Conceptual, Internal levels
Data Independence	Physical and Logical data independence

Table 1: Physical and logical are types of Data Independence.

1.3 Data Abstraction

A Database Management System (DBMS) is designed to efficiently store and retrieve large volumes of data. To simplify user interaction, the DBMS creates a level of abstraction that hides unnecessary details.

1.3.1 Three Levels of Abstraction

1. **View (External) Level:**
   Describes *part* of the database that is relevant to a particular user or application. Different users may have **different views of the same database**. For example, a payroll application may access employee names and salaries, while other users may only see employee names.

2. **Logical (Conceptual) Level:**
   Describes *what* data is stored in the database and the relationships among the data. It includes entities, attributes, relationships, constraints, and **the logical database schema**, while hiding physical storage details.

3. **Physical (Internal) Level:**
   Describes *how* data is **physically stored and accessed**, including file organization, storage structures, indexes, and access paths. These implementation details are hidden from users and applications.

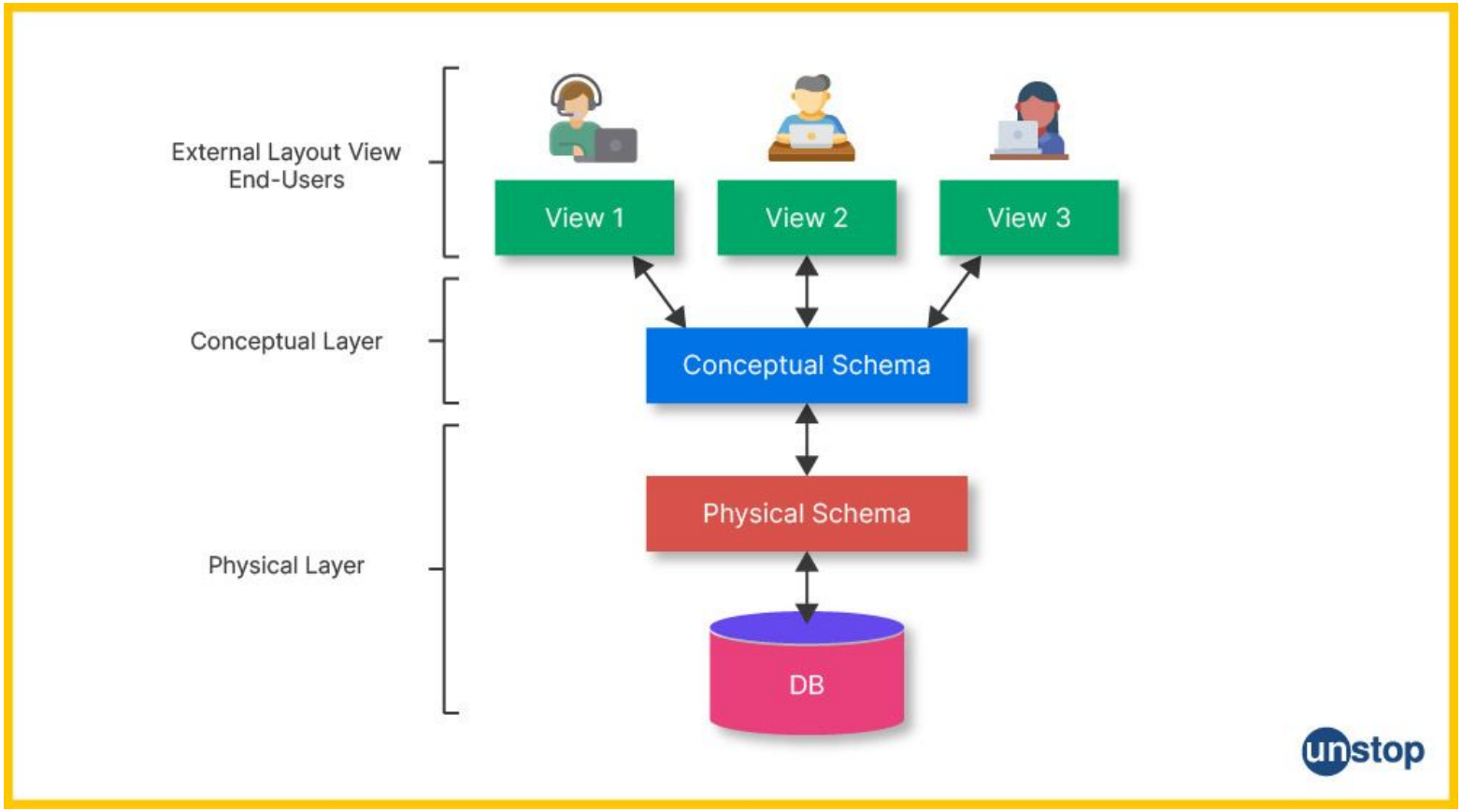


Figure 2: Schema Mappings

Source: Unstop

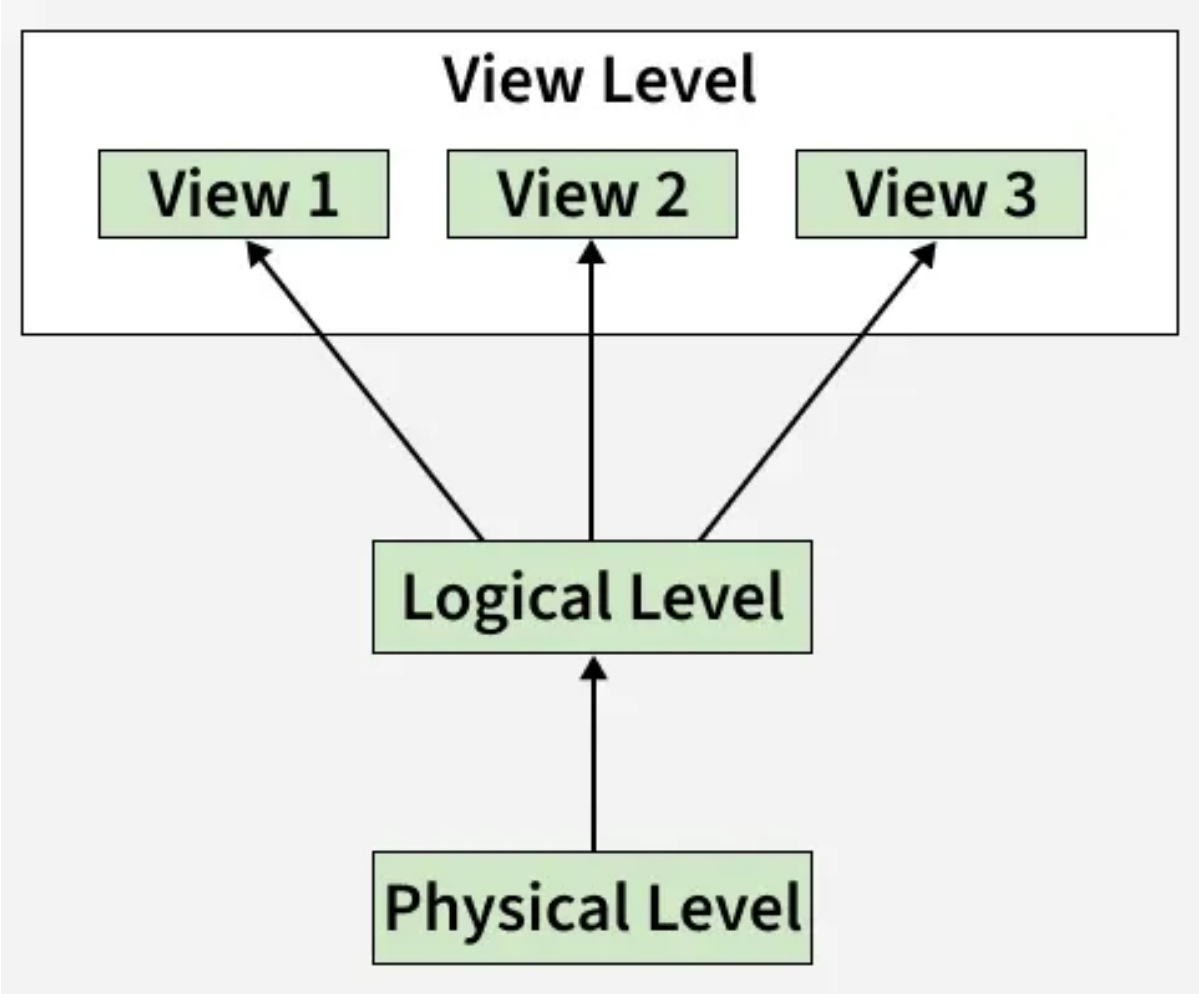


Figure 3: Three Levels of Data Abstraction

Source: GeeksforGeeks

1.4 Data Independence

The ability to modify a schema definition at one level without affecting the schema definition at the next higher level.

- **Physical Data Independence:**
  The ability to change the physical (internal) level without affecting the logical (conceptual) level.
  - *Example:* Switching from sequential files to hash indexing for performance. The SQL tables remain unchanged.

- **Logical Data Independence:**
  The ability to change the logical (conceptual) level without affecting the view (external) level (applications).
  - *Example:* Adding a new column `Email` to the `Customer` table. Existing applications that do not use `Email` continue to work unchanged.

While data abstraction defines *how* we view the data, it does not define *how* we manipulate it. For that, we need a set of fundamental operations.

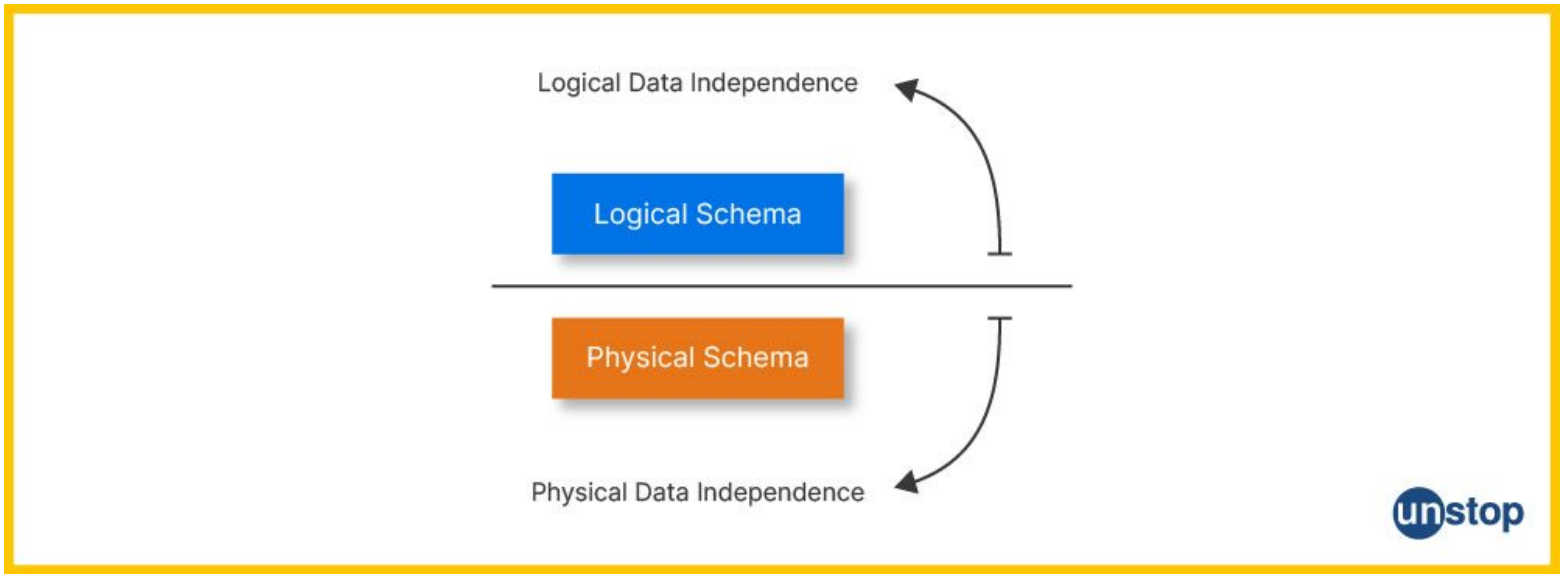


Figure 4: Data Independence Architecture

Source: Unstop

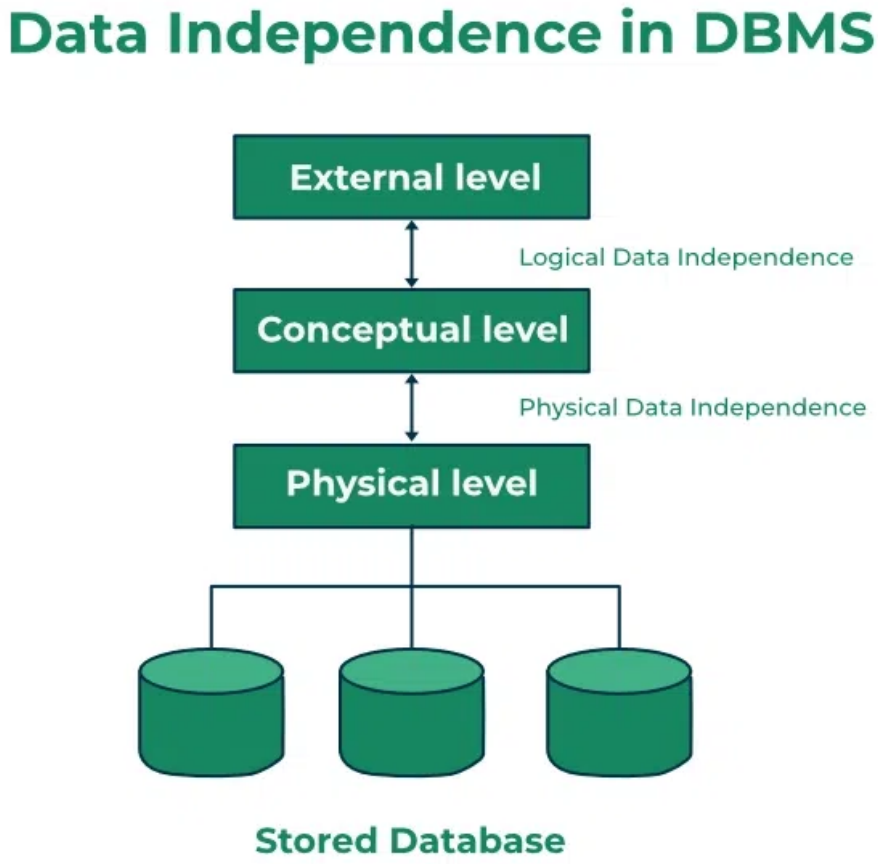


Figure 5: Data Abstraction (1. External/View, 2. Conceptual/Logical, 3. Internal/Physical) and Data Independence (Physical and Logical)

Source: GeeksforGeeks

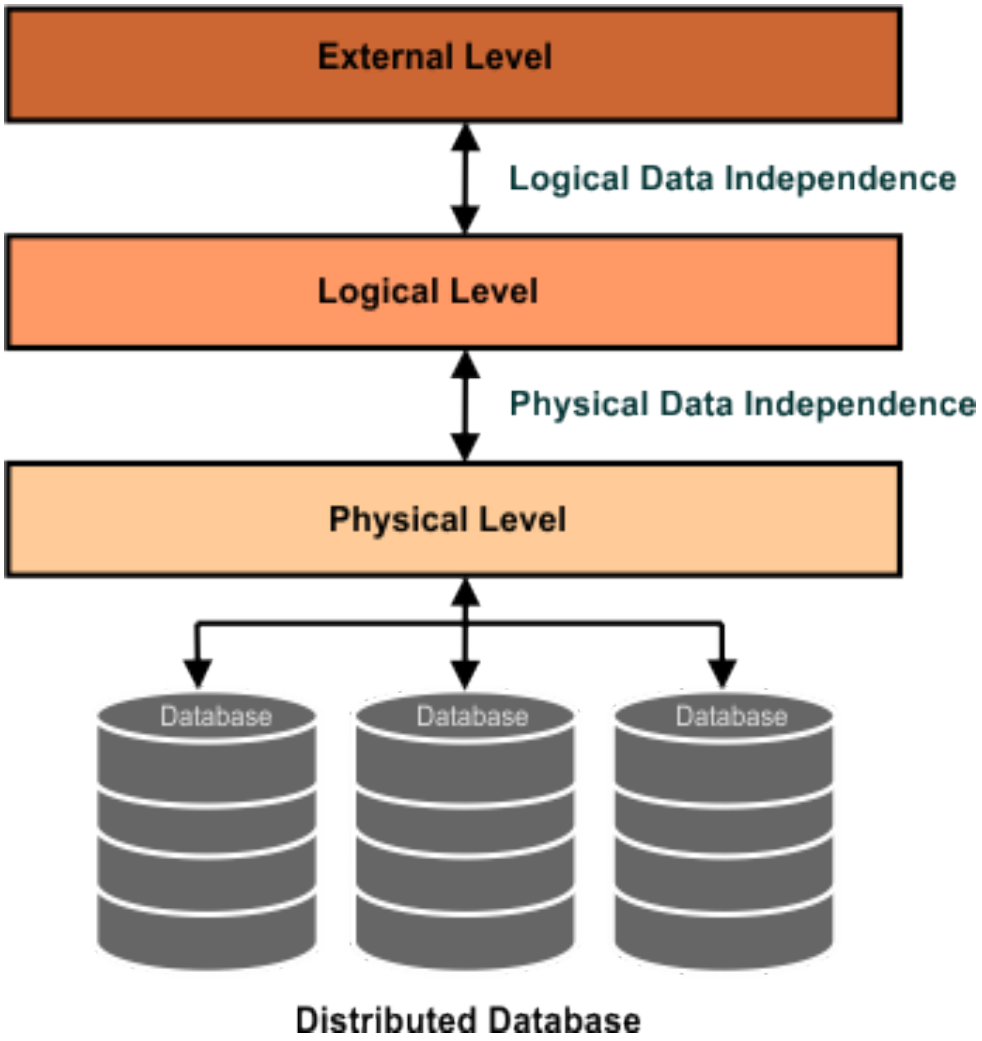


Figure 6: Data Abstraction (1. External/View, 2. Conceptual/Logical, 3. Internal/Physical) and Data Independence (Physical and Logical)

Source: CS Taleem

1.5 Basic Database Operations (CRUD)

Before learning about transactions, we first need to understand four basic operations on data: **CRUD**, which stands for **Create, Read, Update, and Delete**.

- **C - Create:**
  Inserting new data. `INSERT INTO Students (name) VALUES ('Alice');`
- **R - Read:**
  Querying existing data. `SELECT * FROM Students WHERE id = 1;`
  Note: Reads do *not* change the database state.
- **U - Update:**
  Modifying existing data. `UPDATE Students SET name = 'Bob' WHERE id = 1;`
- **D - Delete:**
  Removing data. `DELETE FROM Students WHERE id = 1;`

In complex applications, executing a single CRUD operation is not always enough.

Transactions are simply sequences of these CRUD operations that span multiple steps but need to be treated as a single unit. In reality, real-world tasks usually require executing multiple CRUD statements together (e.g., deducting money from one account AND adding it to another). This leads us to the concept of a Transaction.

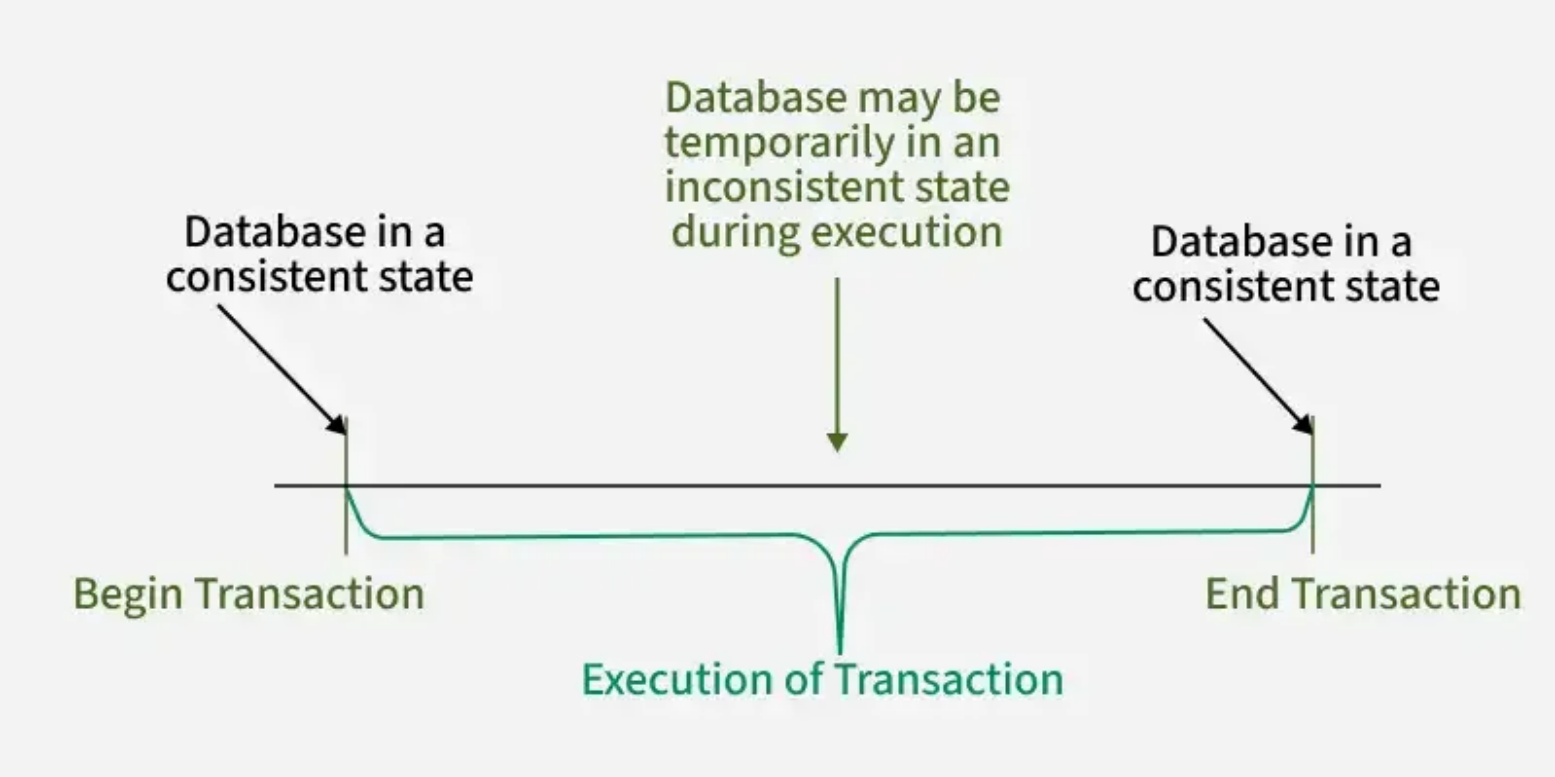


Figure 7: Transaction in DBMS

Source: GeeksforGeeks

1.6 Database Transactions

A **transaction** is a sequence of database operations (reads and writes, via CRUD operations) that performs a single logical unit of work.

The database system (DBMS) ensures that **the transaction is executed as a single logical unit of work**, despite system failures or concurrent users.

Real World Context: Online Ticket Booking

When you buy a concert ticket, multiple steps occur:

1. You select the seat.
2. The system "locks" the seat so no one else can take it.
3. You enter payment details and complete the payment.
4. The system generates the PDF ticket.

If you pay (Step 3) but the system crashes before generating the ticket (Step 4), you would be furious.

A **transaction** ensures that either all of these steps happen, or none of them happen (money is returned).

Example 1: Online Ticket Booking

```sql
BEGIN;

-- Step 1: Select the seat
SELECT *
FROM Seats
WHERE event_id = 101 AND seat_id = 'A12';

-- Step 2: Lock the seat
SELECT *
FROM Seats
WHERE event_id = 101 AND seat_id = 'A12'
FOR UPDATE;

-- Step 3: Complete the payment
INSERT INTO Payments (user_id, event_id, seat_id, amount)
VALUES (1001, 101, 'A12', 2500);

-- Step 4: Generate the ticket
INSERT INTO Tickets (user_id, event_id, seat_id)
VALUES (1001, 101, 'A12');

-- Mark the seat as sold
UPDATE Seats
SET status = 'Sold'
WHERE event_id = 101 AND seat_id = 'A12';

COMMIT;
```

Real World Context: E-Commerce Order placement

Imagine buying the last iPhone in stock:

1. System creates a new **Order #101**.
2. System updates **Inventory** (Product Count: $1 \rightarrow 0$).
3. System charges your **Credit Card**.

The Risk: If the Credit Card is declined (Step 3), the system must **Rollback** Step 1 and Step 2. Otherwise, the iPhone remains "sold" (Inventory = 0) but no one paid for it, and no one else can buy it. A transaction ensures either *Sales + Inventory + Payment* happens together, or nothing happens.

Example 2: E-Commerce Order placement

```
BEGIN;
-- Step 1: Create the order
INSERT INTO Orders (order_id, customer_id, product_id) VALUES (101, 1001, 'iPhone');
-- Step 2: Update inventory
UPDATE Inventory SET quantity = quantity - 1 WHERE product_id = 'iPhone' AND quantity > 0;
-- Step 3: Record the payment
INSERT INTO Payments (order_id, amount, status) VALUES (101, 39900, 'Paid');
COMMIT;

-- If any step fails:
-- ROLLBACK;
-- Undo all changes made since BEGIN and restore the database to its state before the transaction
```

Example 3: Consider a banking transfer of \$100 from account A to account B.

```
BEGIN;
-- Read(A)
UPDATE Accounts SET balance = balance - 100 WHERE account_id = 'A';
-- Read(B)
UPDATE Accounts SET balance = balance + 100 WHERE account_id = 'B';
COMMIT;
```

In a naive system, if the power fails after **`Write(A)`** but before **`Write(B)`**, money vanishes. If another user reads A and B in the middle, they observe an inconsistent state (the total amount of money is incorrect). The DBMS prevents this through the ACID properties.

Example 4: Consider a video game trade: Player 1 gives a "Sword" to Player 2, and Player 2 gives 1000 Gold.

```
BEGIN;
-- Step 1: Player 1 loses Sword
DELETE FROM Inventory WHERE player_id = 1 AND item = 'Sword';
-- Step 2: Player 2 gets Sword
INSERT INTO Inventory (player_id, item) VALUES (2, 'Sword');
-- Step 3: Player 2 pays Gold
UPDATE Wallets SET gold = gold - 1000 WHERE player_id = 2;
-- Step 4: Player 1 gets Gold
UPDATE Wallets SET gold = gold + 1000 WHERE player_id = 1;
COMMIT;
```

Why Atomic? If the server crashes after Step 1, Player 1 loses their sword, but Player 2 never gets it. The item is deleted from existence! Atomicity ensures the swap happens completely or not at all.

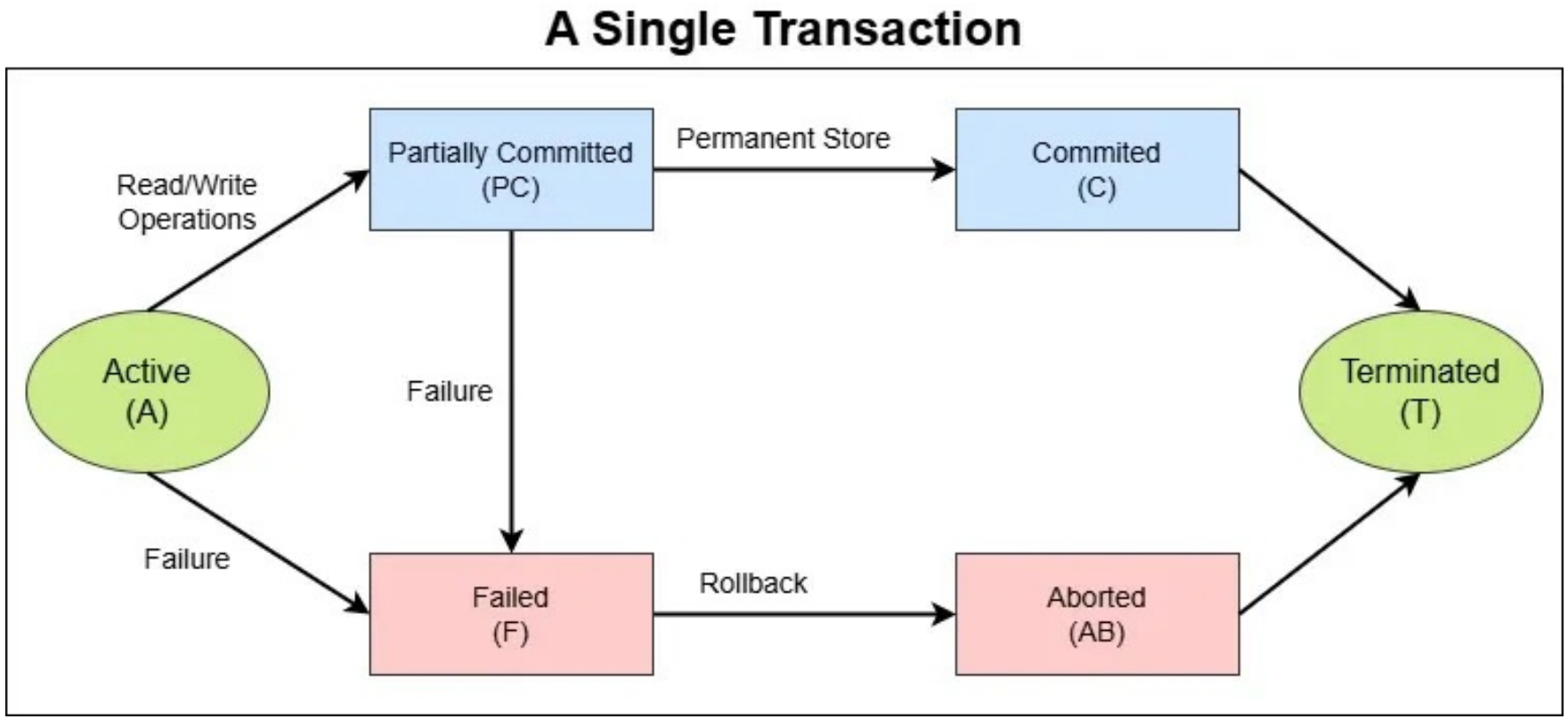


Figure 8: ACID Properties

Source: Medium

1.7 ACID Properties

ACID properties are guarantees provided by the **DBMS** to ensure that transactions are processed reliably and consistently.

- **Atomicity:** All or nothing (e.g., all steps succeed or none).
- **Consistency:** Keeps the database valid (e.g., no negative in quantity).
- **Isolation:** Transactions do not affect each other (e.g., concurrent orders).
- **Durability:** Committed changes are not lost (e.g., after a crash).

Transactions in a relational database are required to satisfy the **ACID** properties. These properties act as safeguards against errors, system crashes, and concurrent access issues, ensuring that data integrity is maintained at all times.

Real World Context: Reliability

Imagine an ATM crashing right after it gives you cash but before it deducts the amount from your balance (the bank loses money).

Real World Context: Reliability

Or imagine two people booking the last flight seat at exactly the same millisecond (double booking).

In short: ACID Properties

ACID properties define the guarantees that a **DBMS** provides for reliable execution of **transactions**.

- **ACID** → Guarantees
- **DBMS** → Provides the guarantees
- **Transactions** → Receive the guarantees

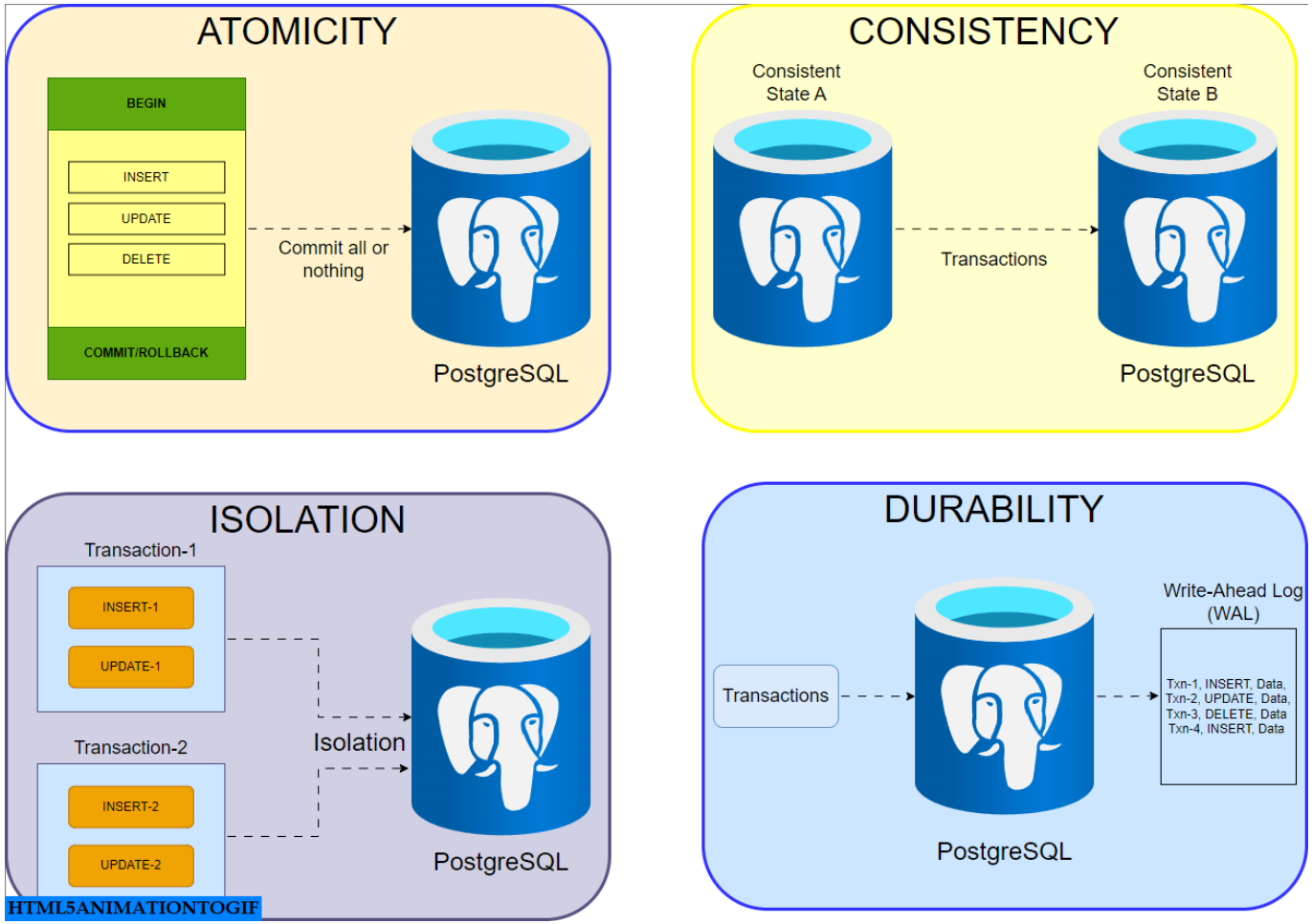


Figure 9: Database ACID Properties

Source: Animesh Gaitonde

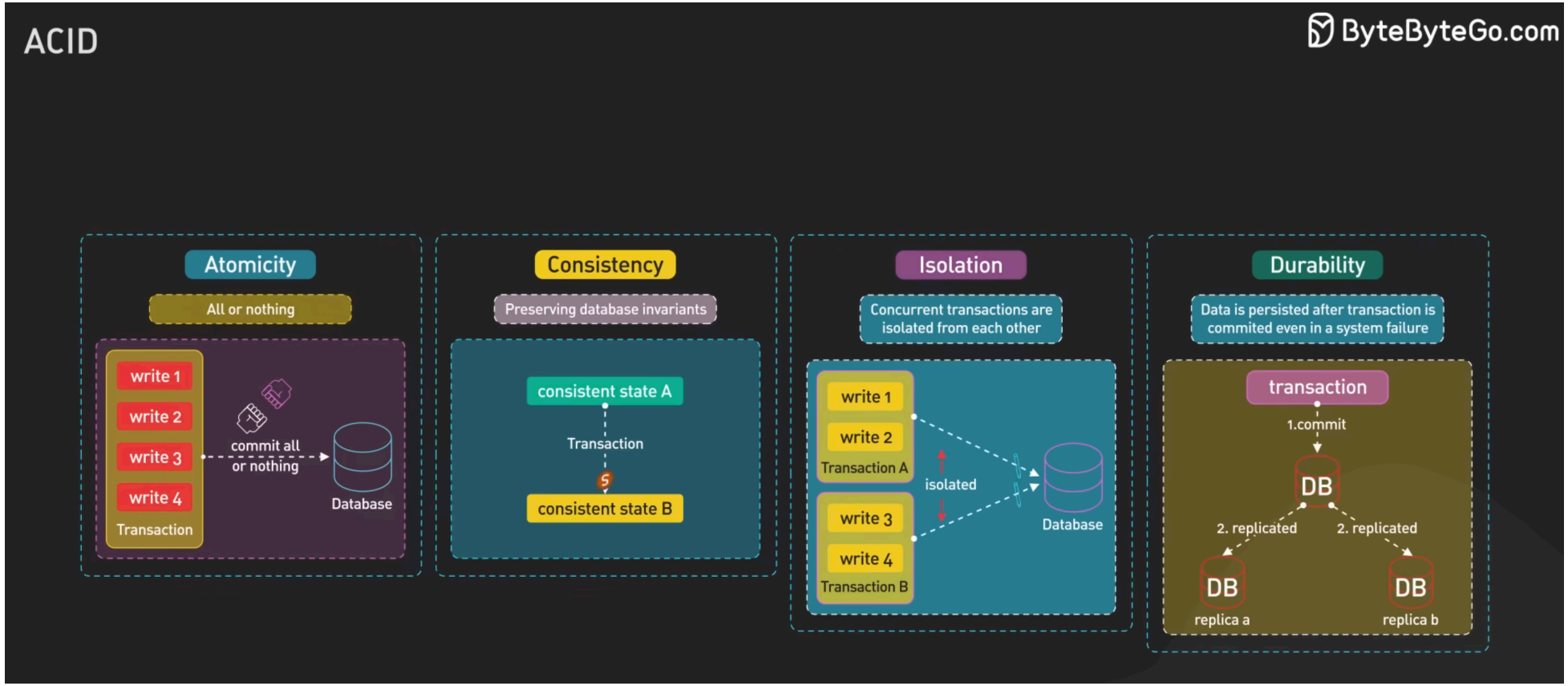


Figure 10: ACID Properties in Databases

Source: Bytebytego.com

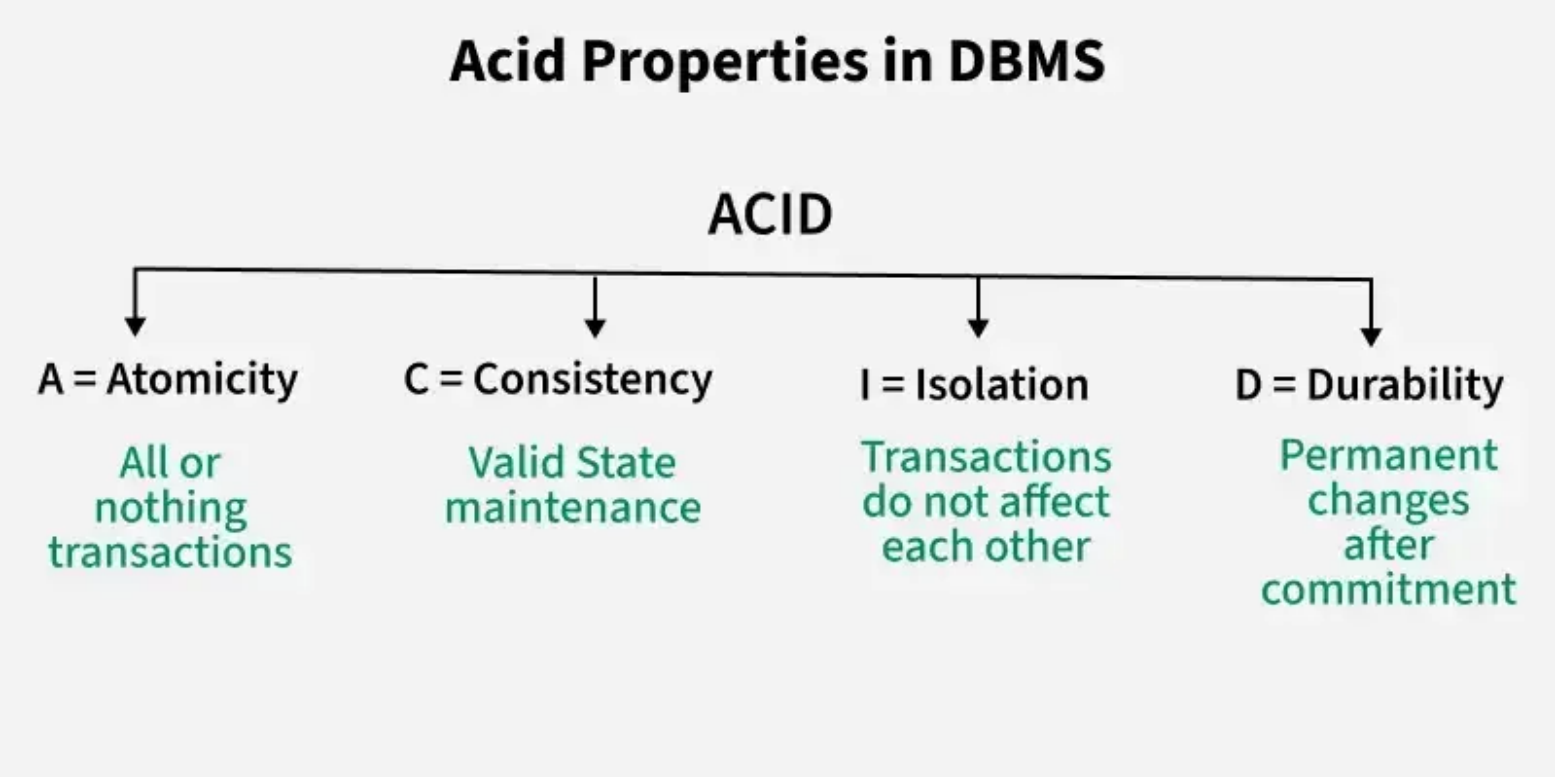


Figure 11: ACID Properties in DBMS

Source: GeeksforGeeks

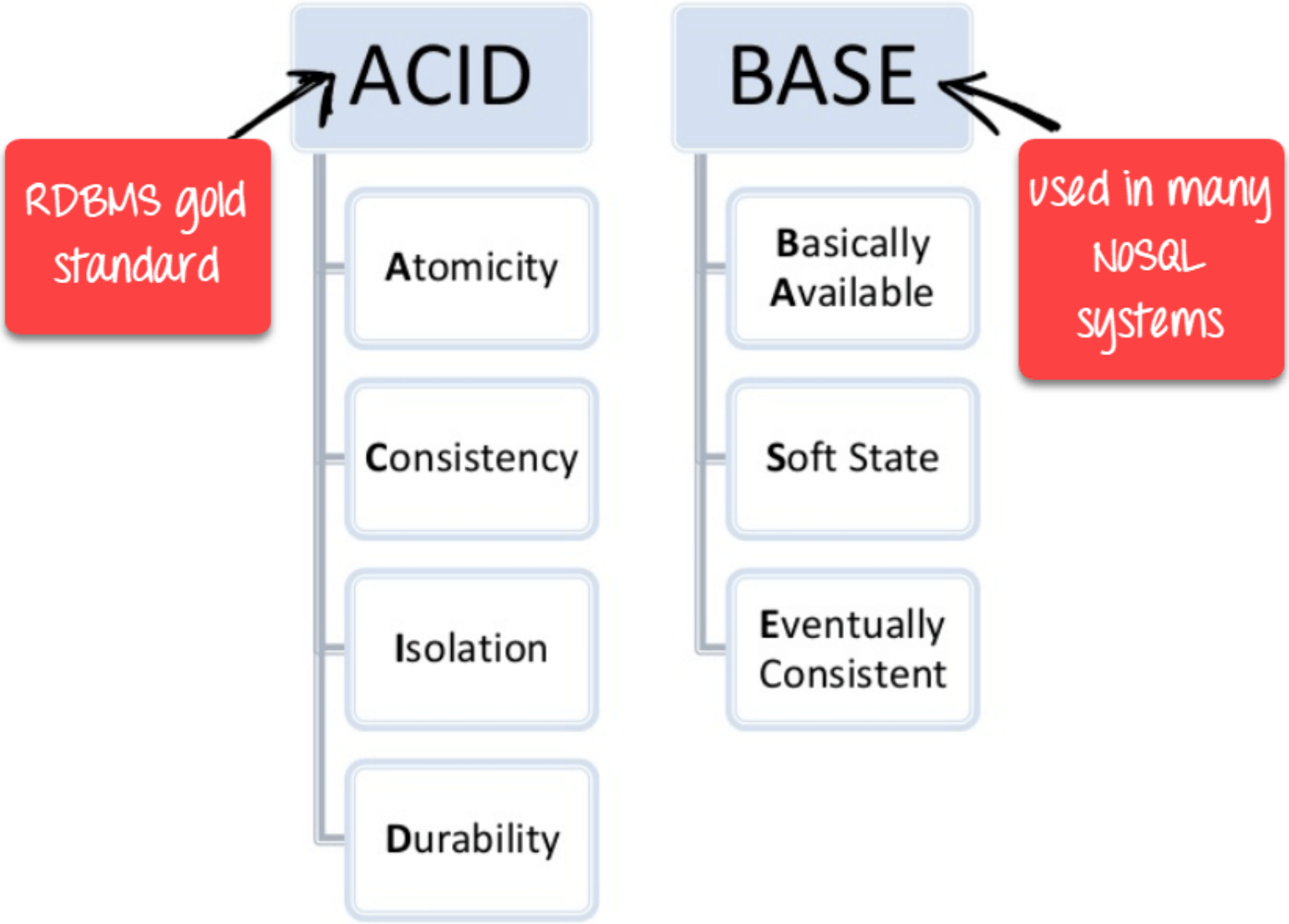


Figure 12: ACID vs BASE Database Properties

Source: LinkedIn - Rudra Narayan Garnaik

1.7.1 Atomicity (All or Nothing)

A transaction must execute completely or not at all. If any part fails, the entire transaction fails.

Concept:

The term "Atomic" comes from the Greek word *atomos*, meaning "indivisible." In the database world, this means a transaction is treated as a **Single Logical Unit of Work**. Even if a transaction contains 10, 100, or 1,000 individual SQL statements, the outside world sees it as one single operation.

The Rule:

- **All or Nothing:**
  Either ALL the changes in the transaction are saved (Committed) , or NONE of them are (Rolled Back).
  - `BEGIN`: Starts the transaction. Sometime may call as `START TRANSACTION` depending on the DBMS.
  - `COMMIT`: Saves all changes.
  - `ROLLBACK`: Manually undoes all changes since `BEGIN`.
- **No Partial Updates:**
  The database must never be left in a "half-done" state, even if the power goes out in the middle of a write operation.

Note

`COMMIT` must be executed to save the changes, while `ROLLBACK` must be executed to undo the changes made by the transaction. Otherwise, the changes remain pending until the transaction is completed.

The exact behavior may different depending on the DBMS and transaction mode.

Real World Context: ATM Withdrawal

Imagine you withdraw $100 from an ATM:

1. The ATM dispenses cash ($100 in your hand).
2. The system deducts $100 from your account balance.

The Risk:
If the system crashes after dispensing cash but before updating your balance, you get $100 for free, and the bank loses money.

Atomicity ensures both steps succeed together or both are undone (cash is returned, balance unchanged).

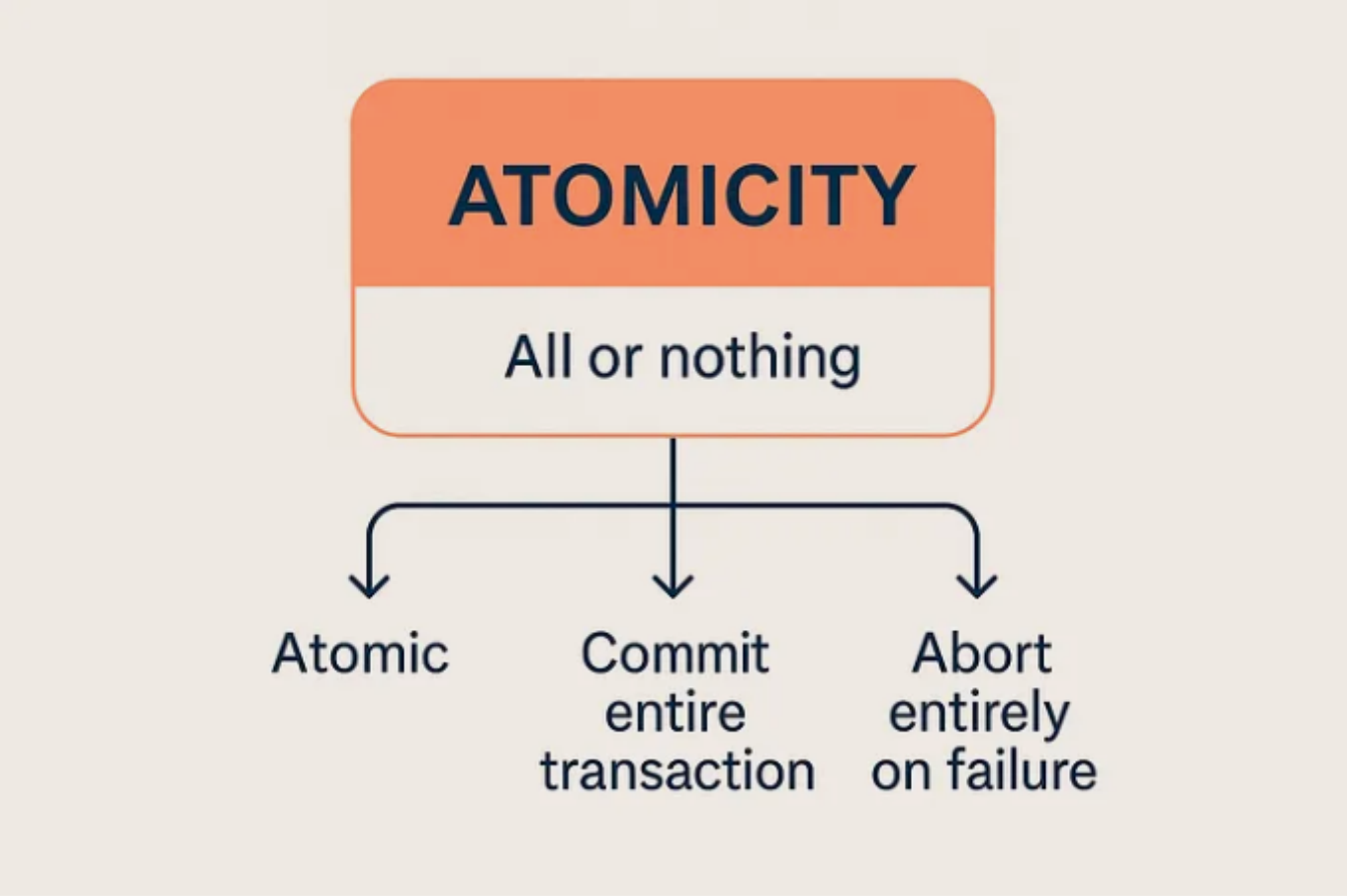


Figure 13: Atomicity: All or Nothing

Source: Kundan Kumar

1. Plain Usage (Without Atomicity)

Without transactions, each SQL statement is independent.

Errors leave data in a "half-done" state (known by humans not DBMS).

An anomaly is a problem that occurs when data is inserted, updated, or deleted.

Anomaly 1: Broken Money Transfer

```
-- Step 1: Deduct from John
UPDATE Balances SET Balance = Balance - 300 WHERE ID = 2;
-- SYSTEM CRASH! (Power failure)
-- Step 2 never runs. Money is destroyed.
```

Anomaly 2: Unpaid Reservation

```
-- Step 1: Reserve seat 1A
UPDATE Seats SET Status = 'Reserved' WHERE SeatNo = '1A';
-- SYSTEM ERROR! (Payment Gateway Time Out)
-- Step 2 (Payment) never happens. Seat is blocked forever.
```

2. With Atomicity: Success (Commit)

We wrap statements in a transaction. `COMMIT` saves everything at once.

Example 1: Successful Transfer

```
-- BEGIN;              -- Shorter; commonly used in PostgreSQL/SQLite
-- START TRANSACTION;  -- Standard SQL-style; more explicit

START TRANSACTION;
UPDATE Balances SET Balance = Balance - 300 WHERE ID = 2;
UPDATE Balances SET Balance = Balance + 300 WHERE ID = 1;
COMMIT; -- Both changes saved together.
```

Example 2: Confirmed Booking

```
START TRANSACTION;
UPDATE Seats SET Status = 'Reserved' WHERE SeatNo = '1A';
INSERT INTO Payments (Amount, SeatNo) VALUES (500, '1A');
COMMIT; -- Seat and Payment saved together.
```

3. With Atomicity: Failure (Rollback)

If an error occurs, we explicitly `ROLLBACK` to undo everything.

Example 1: Safe Failure (Transfer)

```
START TRANSACTION;

UPDATE Balances SET Balance = Balance - 300 WHERE ID = 2;
-- Error: The database cannot update the balance because the account ID does not exist.

ROLLBACK;
-- John's balance is restored to its state before the transaction.
-- Should call COMMIT; but no COMMIT occurs, so the balance change is undone.
```

Example 2: Safe Failure (Booking)

```
START TRANSACTION;

UPDATE Seats SET Status = 'Reserved' WHERE SeatNo = '1A';
-- Error: The database cannot reserve the seat because it has already been reserved.

ROLLBACK;
-- Seat '1A' remains in its previous state.
-- Should call COMMIT; but no COMMIT occurs, so the seat reservation is undone.
```

In BDMS detail:

Atomicity is guaranteed using **Undo Logs** (part of WAL). Before a transaction changes any data, the DBMS writes a log record containing the *old value*.

- If the transaction fails, the Recovery Manager reads the log and uses these old values to **undo** the changes.
- This ensures that no partial updates ever persist in the database.

1.7.2 Consistency (Correctness)

A transaction must transform the database from one valid state to another valid state, **preserving all integrity constraints (valid-to-valid)** defined by the database schema.

These rules can be simple ("Balance cannot be negative") or complex ("A student cannot enroll in more than 5 courses per semester"). The DBMS acts as a **Rule Enforcer**, rejecting any transaction that violates these constraints.

Example: If the schema requires that an account balance cannot be negative, a transaction must not leave the database with a negative balance, even when multiple transactions execute concurrently.

Concept:
Consistency means the database remains valid according to its defined rules and constraints.

The Rule:

- **Valid State → Valid State:**
  A transaction can only move the database from one valid state to another (valid-to-valid). It cannot leave the database in a broken state.
- **Constraints Always Hold:**
  Primary keys must be unique, foreign keys must reference existing rows, and CHECK constraints must be satisfied.

Real World Context: Age Verification

A university enrollment system has a rule: *"Students must be at least 18 years old."*

A transaction attempts to insert:

- Name: "Alice"
- Age: 16 (violates the rule!)

Without Consistency: Alice is enrolled despite being underage. The database is now in an *invalid state*.
With Consistency: The DBMS rejects the transaction with an error: `"CHECK constraint failed: Age >= 18"`. The database remains valid.

Example: Integrity Constraints

A transaction attempts to set `Balance = -100` or insert a duplicate account ID.

- Constraint 1: Balance cannot be negative.
- Constraint 2: Account IDs must be unique.

Without consistency: The DBMS allows a negative balance or a duplicate ID. Business rules are violated.
With Consistency: The DBMS continually checks constraints.
If a transaction violates a constraint (e.g., `CHECK (Balance >= 0)`), the DBMS aborts the transaction immediately.

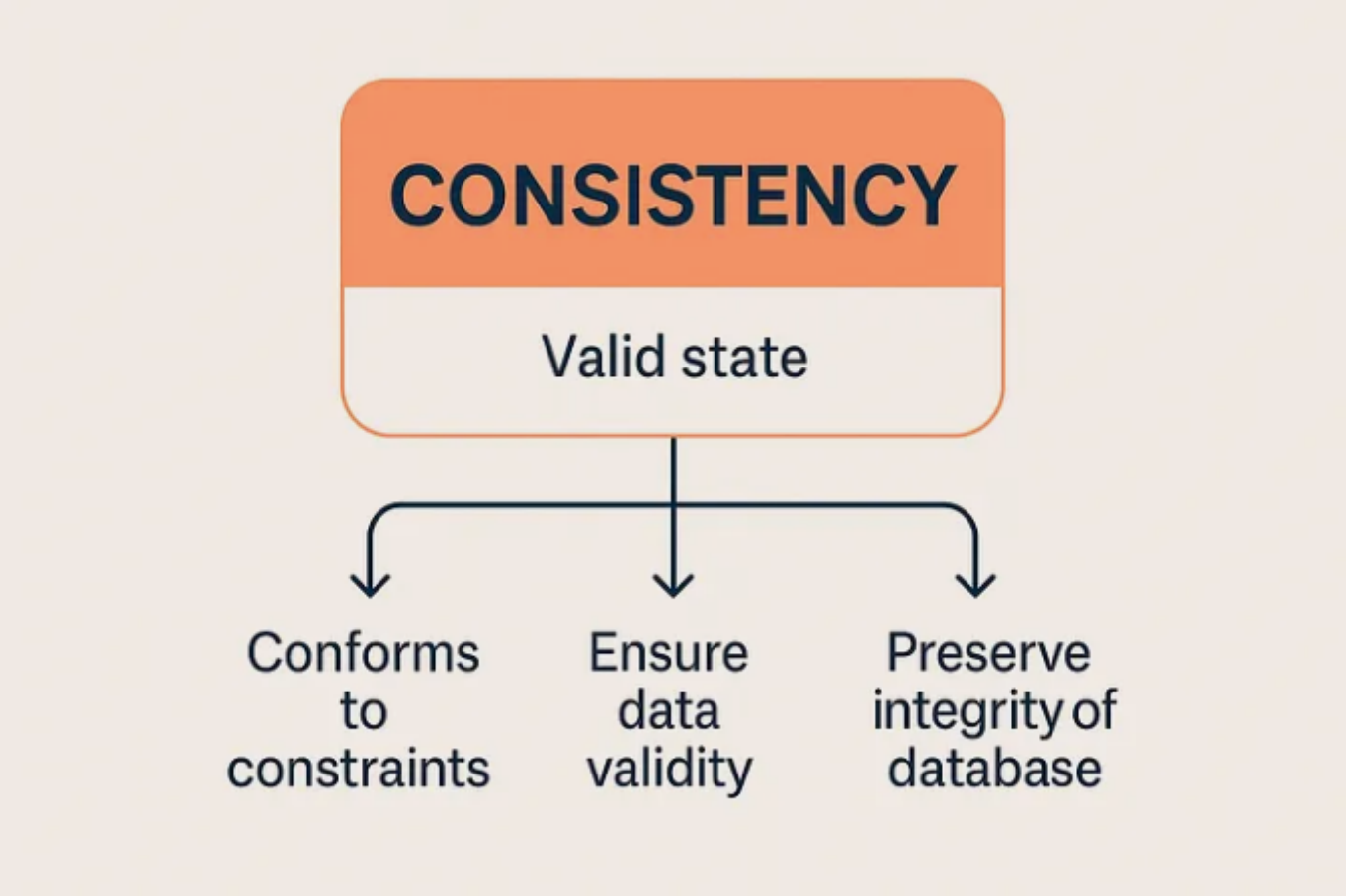


Figure 14: Consistency: Valid State

Source: Kundan Kumar

How Consistency Is Enforced:

The DBMS enforces the constraints defined in the database schema to keep the database valid.

1. Schema Constraints:

Rules defined in the database schema during database design and enforced by the DBMS, e.g., `CHECK`, `PRIMARY KEY`, `FOREIGN KEY`, and `UNIQUE`.

```sql
-- CHECK Constraint: defines valid values
CREATE TABLE Accounts (

    account_id INT PRIMARY KEY,
    balance DECIMAL(10,2) CHECK (balance >= 0),
    account_type VARCHAR(20) CHECK (account_type IN ('Savings', 'Checking'))

);

-- FOREIGN KEY Constraint: defines valid relationships
ALTER TABLE Accounts
ADD CONSTRAINT fk_customer
FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
ON DELETE CASCADE;

-- UNIQUE Constraint: prevents duplicate values
ALTER TABLE Customers
ADD CONSTRAINT unique_email UNIQUE (email);
```

2. Triggers:

Procedures automatically executed by the DBMS when specified database events occur, such as `INSERT`, `UPDATE`, or `DELETE`.

`TRIGGER` is an advanced DBMS feature that automatically executes predefined actions when specified database events occur, such as `INSERT`, `UPDATE`, or `DELETE`.

```
-- Trigger: Prevent overdraft (withdraw amount > balance)
CREATE TRIGGER prevent_overdraft
BEFORE UPDATE ON Accounts
-- FOR EACH ROW: Run the trigger once for each affected row
FOR EACH ROW
BEGIN
    IF NEW.balance < 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Overdraft not allowed';
    END IF;
END;

-- Trigger: Record balance changes
CREATE TRIGGER audit_balance_change
AFTER UPDATE ON Accounts
FOR EACH ROW
BEGIN
    IF NOT (OLD.balance <=> NEW.balance) THEN
        INSERT INTO AuditLogs(account_id, old_balance, new_balance)
        VALUES (OLD.account_id, OLD.balance, NEW.balance);
    END IF;
END;
```

The syntax for defining a `TRIGGER` may differ between DBMSs.

You may not need to focus on the SQL syntax yet; focus on the idea that the DBMS can automatically execute predefined logic when a database event occurs.

Note

Consistency can be enforced by both the *application logic* (application layer) or the *DBMS* (database layer), using mechanisms such as constraints and triggers.

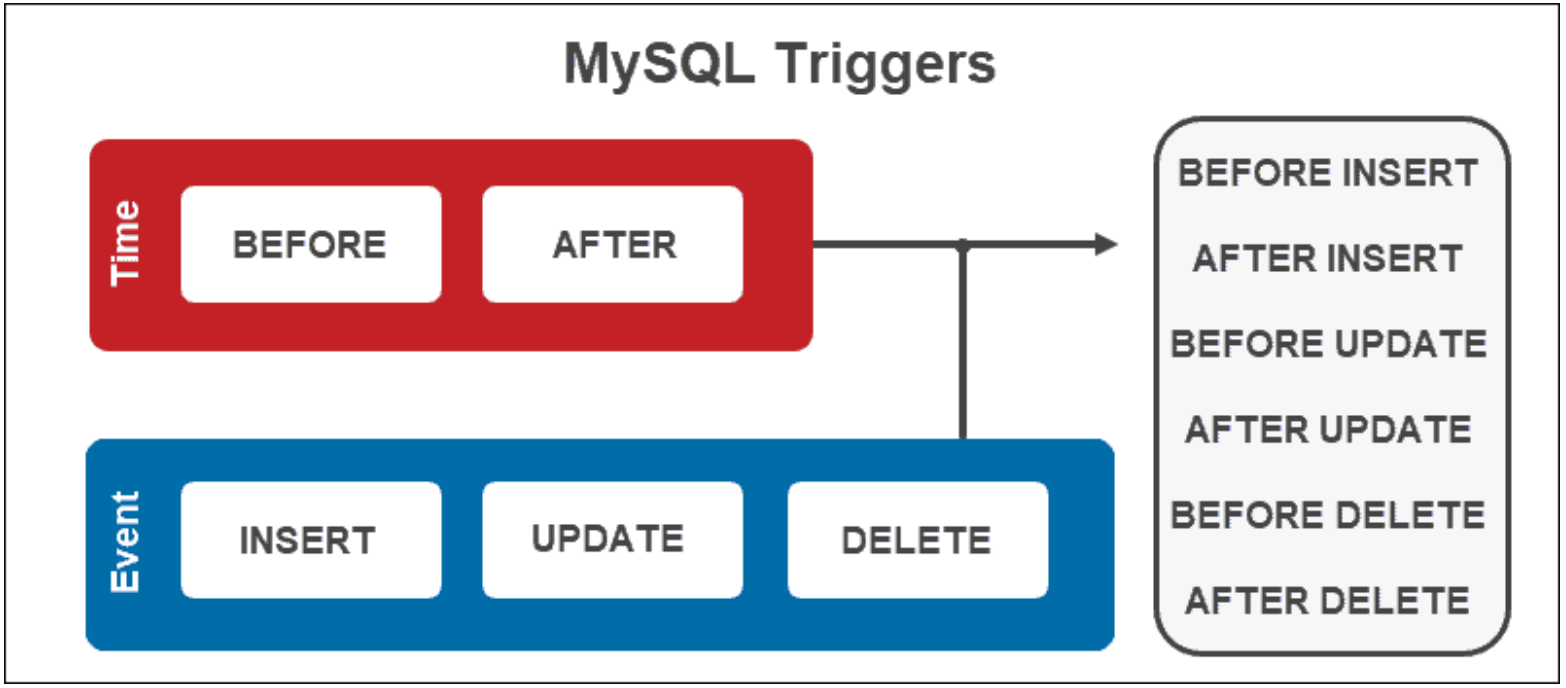


Figure 15: MySQL Triggers

Source: phoenixNAP

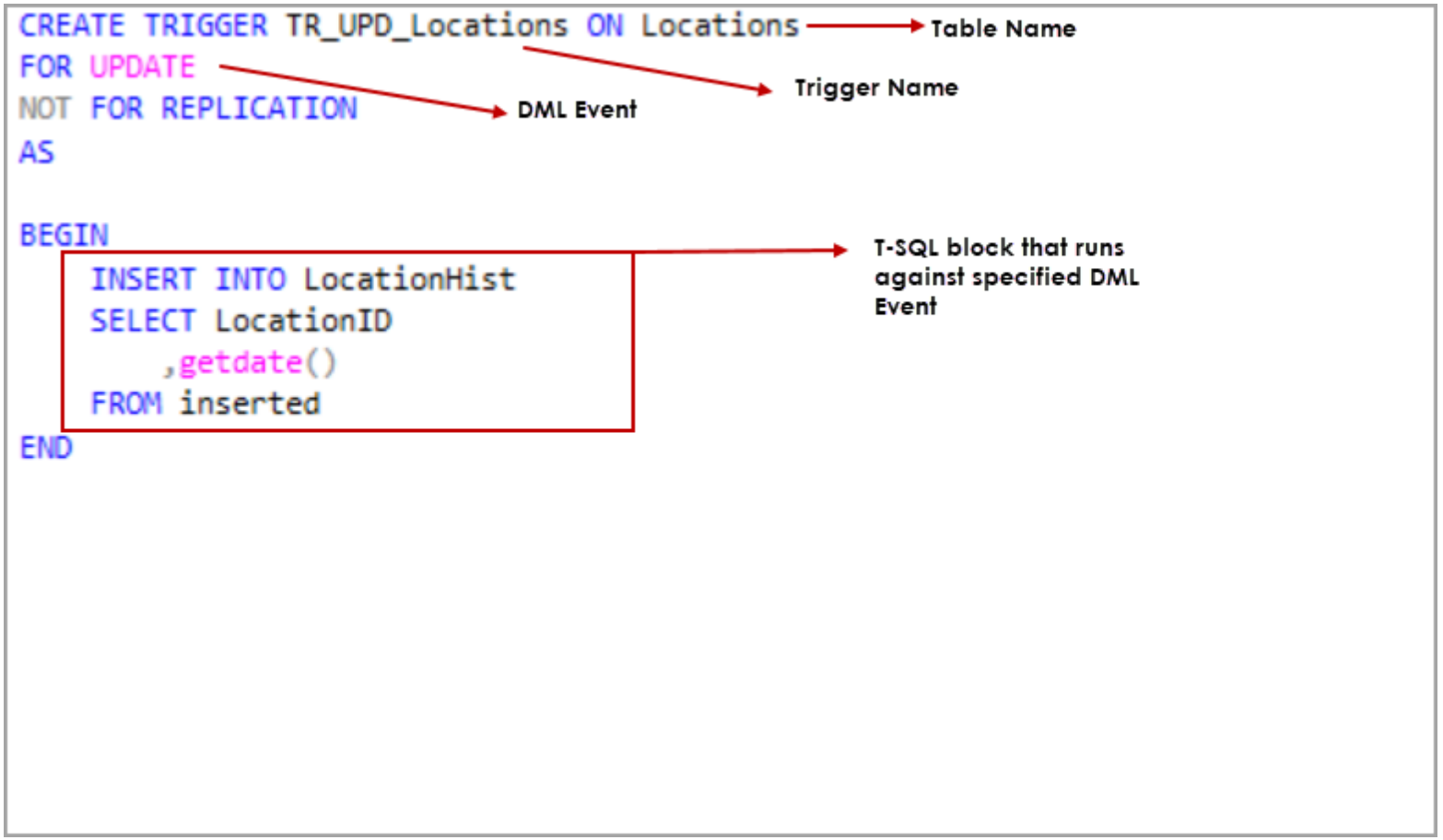


Figure 16: DML Trigger for Update Event (SQL Server)

Source: SQL Shack

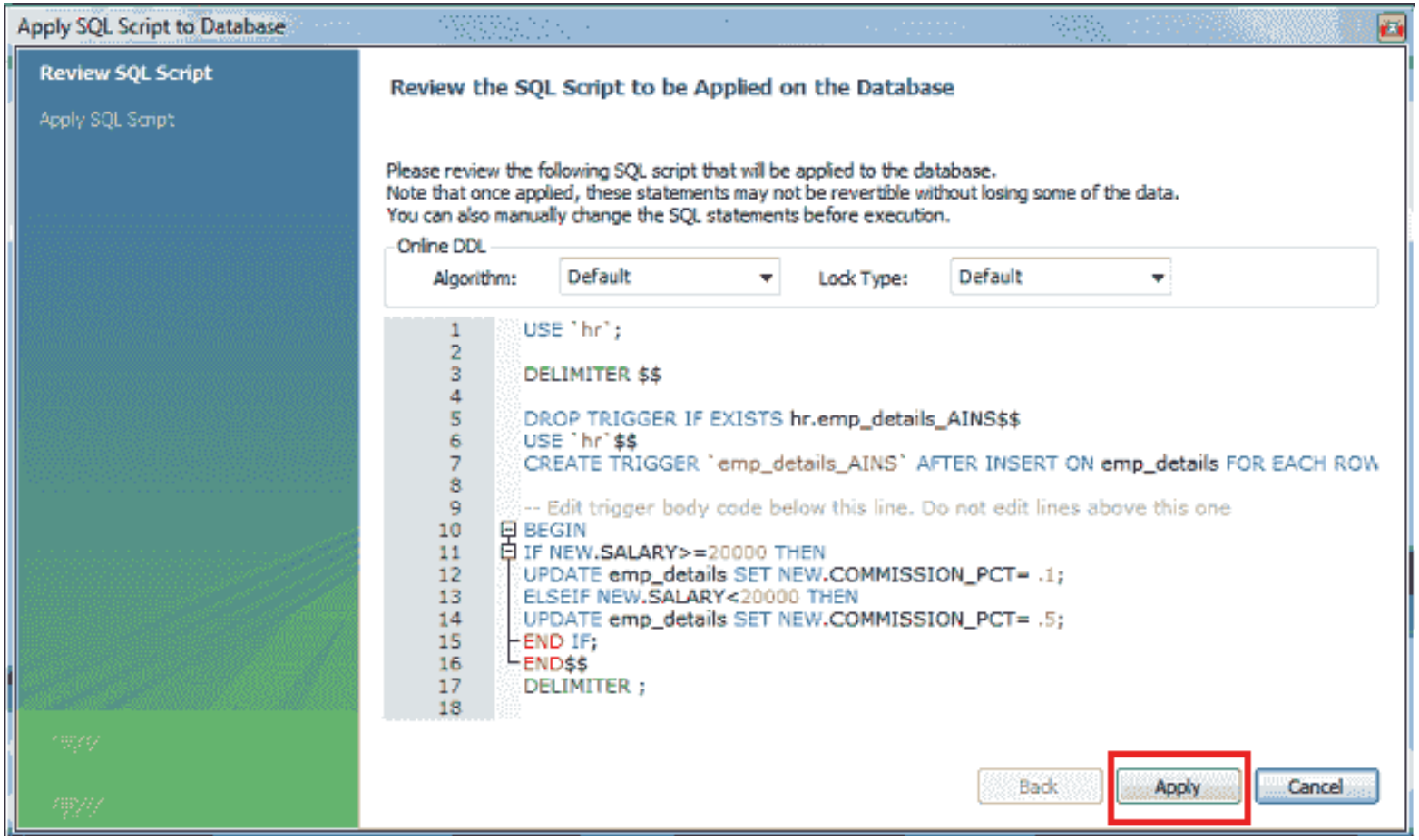


Figure 17: MySQL Workbench Trigger

Source: W3Resource

1.7.3 Isolation (Independence)

Isolation ensures that concurrent transactions do not affect each other in a way that produces incorrect results.

The DBMS uses **Concurrency Control** mechanisms, such as **Locks** and **Multi-Version Concurrency Control (MVCC)**, to provide this isolation.

Example: When two users try to buy the last available seat at the same time (e.g., concert ticket, flight seat, hotel room), the database must ensure that only one transaction can reserve the seat.

Concept:
Isolation means that even if 1,000 transactions are running at the same time, each one behaves as if it is the *only* transaction in the system. Isolation ensures that concurrent transactions do not see or interfere with each other's intermediate changes. This prevents transactions from using incomplete or inconsistent data.

The Rule:

- **Concurrent = Serial:**
  Running transactions concurrently must produce the same result as running them one after another (serializability).
- **No Interference:**
  One transaction's uncommitted changes must not be visible to other transactions.

Real World Context: Concert Ticket Sales

Two people (User A and User B) try to buy the **last remaining seat** for a concert at exactly the same time:

- Time 0: Both see "1 seat available."
- User A: Clicks "Buy" → System processes payment.
- User B: Clicks "Buy" → System processes payment.

Without Isolation: Both payments succeed. Now 2 people have tickets for 1 seat (double booking!).
With Isolation: User A's transaction locks the seat. User B waits. After A commits, B sees "Sold Out" and cannot purchase.

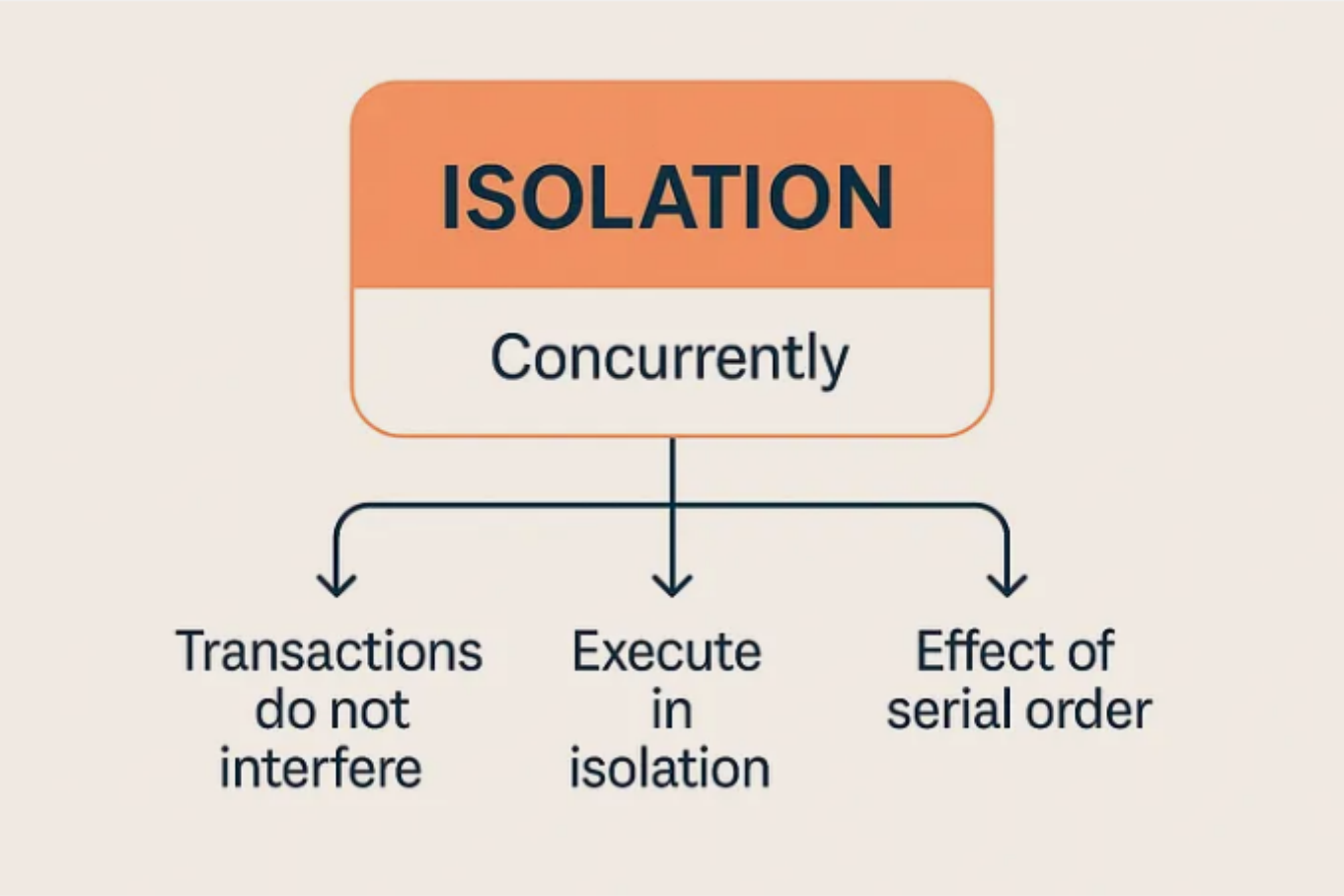


Figure 18: Isolation: Concurrently

Source: Kundan Kumar

Key Isolation Concepts:

- **Serial Schedule:**
  Transactions are executed one at a time, without overlapping. For example, T1 completes before T2 starts.

- **Concurrent Schedule:**
  Operations from multiple transactions are interleaved and may execute at the same time. This improves performance but requires concurrency control.

- **Serializability:**
  A concurrent schedule is *serializable* if its final result is the same as the result of executing the same transactions one at a time in some serial order.

- **Race Condition:**
  A problem that occurs when the result of concurrent transactions depends on the order in which their operations are executed.

Example: Concurrent Updates

- **Setup:** Initial Balance = \$600.
- **Intention T1:** Withdraw \$100. (Expected: $600 - 100 = 500$)
- **Intention T2:** Deposit \$200. (Expected: $500 + 200 = 700$)
- **Correct Final State:** \$700.

Without Concurrency Control: Lost Update

Consider two concurrent transactions operating on the same account. The initial balance is \$600.

```
-- Transaction T1: Withdraw $100

SELECT balance FROM Accounts WHERE id = 1; -- Returns 600

-- T1 calculates: 600 - 100 = 500
UPDATE Accounts SET balance = 500 WHERE id = 1;
```

```
-- Transaction T2: Deposit $200

SELECT balance FROM Accounts WHERE id = 1; -- Returns 600

-- T2 calculates: 600 + 200 = 800
UPDATE Accounts SET balance = 800 WHERE id = 1;
```

Concurrent Execution:

```
-- Initial Balance: $600

-- Step 1: T1 reads the balance
SELECT balance FROM Accounts WHERE id = 1; -- Returns 600

-- Step 2: T2 reads the balance before T1 updates it
SELECT balance FROM Accounts WHERE id = 1; -- Returns 600

-- Step 3: T1 withdraws $100
-- T1 calculates: 600 - 100 = 500
UPDATE Accounts SET balance = 500 WHERE id = 1;

-- Step 4: T2 deposits $200
-- T2 calculates: 600 + 200 = 800
-- T2 overwrites T1's update
UPDATE Accounts SET balance = 800 WHERE id = 1;

-- Final Balance: $800
-- T1's $100 withdrawal is lost.
-- Expected Balance: $600 - $100 + $200 = $700
```

Step 1: T1 reads the balance of $600.

Step 2: T2 also reads $600 before T1's update is applied.

Step 3: T1 subtracts $100 and updates the balance to $500.

Step 4: T2 uses its previously read value of $600, adds $200, and updates the balance to $800. This overwrites T1's update.

Problem (Without transaction): Lost Update

Both transactions read the same old value and then write their own results. The later update overwrites the earlier update, causing T1's withdrawal to be lost.

With Concurrency Control Using Locks

The DBMS can use a lock to prevent concurrent transactions from modifying the same row at the same time.

T1:

```
START TRANSACTION;

-- Lock the row for update
SELECT balance FROM Accounts WHERE id = 1 FOR UPDATE;
UPDATE Accounts SET balance = balance - 100 WHERE id = 1;

COMMIT;
```

T2:

```
START TRANSACTION;

-- Waits if T1 still holds the lock
SELECT balance FROM Accounts WHERE id = 1 FOR UPDATE;
UPDATE Accounts SET balance = balance + 200 WHERE id = 1;

COMMIT;
```

Execution Flow with Locking:

```
-- Initial Balance: $600
-- Step 1: T1 starts and locks the row
START TRANSACTION;
SELECT balance FROM Accounts WHERE id = 1 FOR UPDATE; -- Returns 600 and locks the row

-- Step 2: T2 tries to lock the same row
START TRANSACTION;
SELECT balance FROM Accounts WHERE id = 1 FOR UPDATE;
-- T2 is blocked and waits for T1 to release the lock.

-- Step 3: T1 withdraws $100
UPDATE Accounts SET balance = balance - 100 WHERE id = 1;

COMMIT; -- T1 commits and releases the lock

-- Step 4: T2 acquires the lock and reads the updated balance, T2 now sees $500
UPDATE Accounts SET balance = balance + 200 WHERE id = 1;

COMMIT; -- T2 commits and releases the lock
-- Final Balance: $700
```

Step 1: T1 locks the row using `FOR UPDATE`.

Step 2: T2 attempts to lock the same row but must wait because T1 currently holds the lock.

Step 3: T1 subtracts $100 and commits. The balance becomes $500, and the lock is released.

Step 4: T2 acquires the lock and operates on the updated balance of $500. It adds $200, resulting in a final balance of $700.

Result (with transaction)

Concurrency control prevents the lost update by ensuring that conflicting transactions do not modify the same data at the same time. The final balance is $700, which correctly reflects both transactions.

Why Concurrency Control Is Needed:

Without proper concurrency control, concurrent transactions may produce incorrect results or encounter anomalies such as **Dirty Reads**, **Lost Updates**, and **Phantom Reads (wrong results)**.

How the DBMS Provides Isolation:

Mechanism:

The DBMS uses **Concurrency Control** to manage concurrent transactions and enforce Isolation.

Common Techniques:

Two widely used approaches are:

- **Two-Phase Locking (2PL):**
  Transactions acquire locks before accessing data. If a required lock is held by another transaction, the transaction must wait until the lock is released. 2PL can provide serializable execution, but it may lead to **deadlocks** when transactions wait for each other.
- **Multi-Version Concurrency Control (MVCC):**
  ˜ The DBMS keeps multiple versions of data. A transaction can read an older version while another transaction is updating a newer version. This allows read and write operations to proceed concurrently without always blocking each other.

The goal is to ensure that concurrent execution is equivalent to an appropriate serial execution, providing **Serializability** where required by the isolation level.

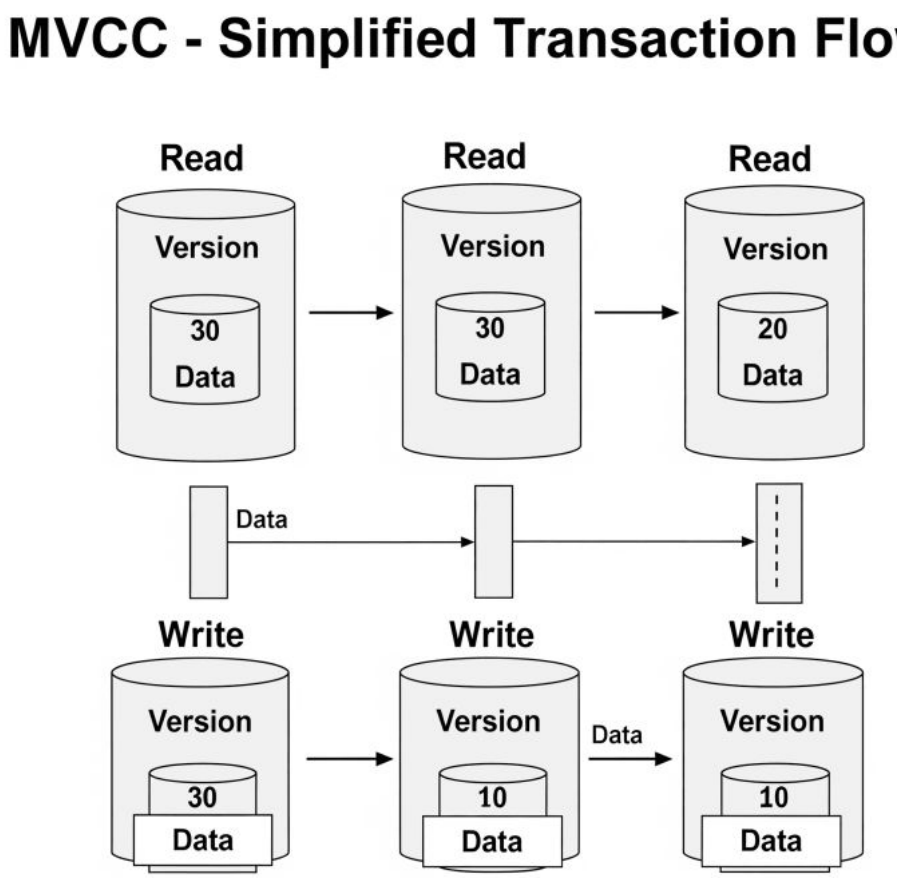


Figure 19: Transaction Isolation Levels

Source: Transaction Isolation Levels

1.7.3.1 Transaction Isolation Levels

An **isolation level** defines how much a transaction is isolated from the changes made by other concurrent transactions.

Higher isolation generally provides stronger protection against concurrency anomalies, but may reduce concurrency or performance.

Four Standard Isolation Levels:

1. **Read Uncommitted:**
   A transaction may read changes made by other transactions before they are committed.
2. **Read Committed:**
   A transaction can read only data that has been committed. Dirty reads are prevented.
3. **Repeatable Read:**
   Data already read by a transaction remains consistent when read again during the same transaction.
4. **Serializable:**
   The strongest standard isolation level. Concurrent transactions produce a result equivalent to some serial execution.

Isolation Level Summary Table:

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Lost Update
Read Uncommitted	Possible	Possible	Possible	Possible
Read Committed	Prevented	Possible	Possible	Possible
Repeatable Read	Prevented	Prevented	Possible	Prevented
Serializable	Prevented	Prevented	Prevented	Prevented

Table 2: Transaction Isolation Levels and Anomaly Prevention

Note

- The default isolation level may differ between DBMSs, so the behavior of concurrent transactions can vary between database systems.
- The isolation level can also be specified for each transaction when needed.

1.7.4 Durability (Persistence)

Once a transaction is successfully committed, its changes must survive subsequent system failures, such as a system crash or power loss.

Example: A user transfers $500 from their bank account and the database confirms the transaction. If the server loses power immediately afterward, the transfer must not be lost. After recovery, the database must still reflect the completed transfer.

Concept:
Durability means that once the database tells you "Success," your data is **permanent**. Even if the server catches fire immediately after, your committed transaction will survive. This is achieved by writing changes to *non-volatile storage* (hard disk/SSD) using **Write-Ahead Logging (WAL)** before acknowledging success.

The Rule:

- **Commit = Permanent:**
  Once a transaction commits, its changes must survive any subsequent failure (crash, power loss, disk failure with backups).
- **No Lost Commits:**
  The database guarantees that committed data is written to stable storage before returning success to the user.

Example: Durability

```
START TRANSACTION;
UPDATE Accounts SET balance = balance - 500 WHERE id = 1;
COMMIT;

-- The DBMS reports that the transaction was successful.

-- If the server crashes immediately after COMMIT,
-- the $500 deduction must not be lost.

-- After recovery, the database still contains
-- the committed balance change.
```

Real World Context: Online Purchase Confirmation

You just bought a $500 laptop online:

1. You enter payment details.
2. The system shows: **"Payment Successful! Order #12345 Confirmed."**
3. *Immediately after*, the server crashes (power outage).

Without Durability: After reboot, your order is gone. You were charged $500 but have no record of the purchase.
With Durability: The transaction was written to disk *before* showing "Success." After reboot, Order #12345 is still in the database, and your laptop will be shipped.

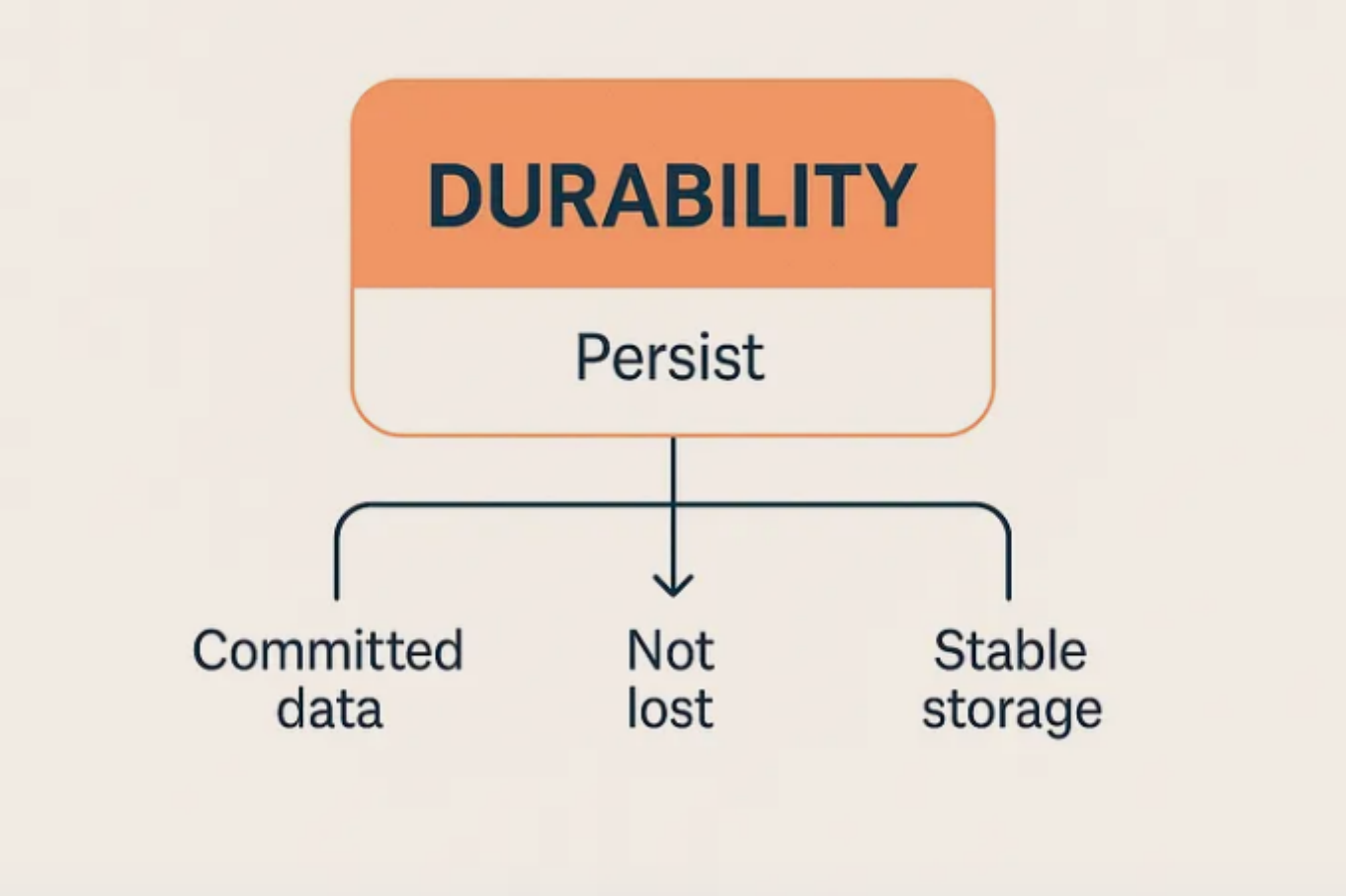


Figure 20: Durability: Persist

Source: Kundan Kumar

Example: Crash after Commit

```
BEGIN;
UPDATE Accounts SET balance = 900 WHERE account_id = 1;
COMMIT;
```

System crashes immediately after sending "Success" to the user.

Without Durability: The change was only in RAM (volatile). On reboot, the balance is back to 600.
With Durability: Before sending "Success", the DBMS writes the change to the **Write-Ahead Log (WAL)** on disk.

- On reboot, the Recovery Manager reads the WAL and reapplies the change.

Implementation:

Durability is supported by **Write-Ahead Logging (WAL)**. Before the DBMS confirms a successful `COMMIT`, the log records describing the transaction's changes are written to reliable, non-volatile storage. If the system crashes, **the Recovery Manager uses the log to identify committed changes and redo them during recovery**, ensuring that the committed changes are not lost.

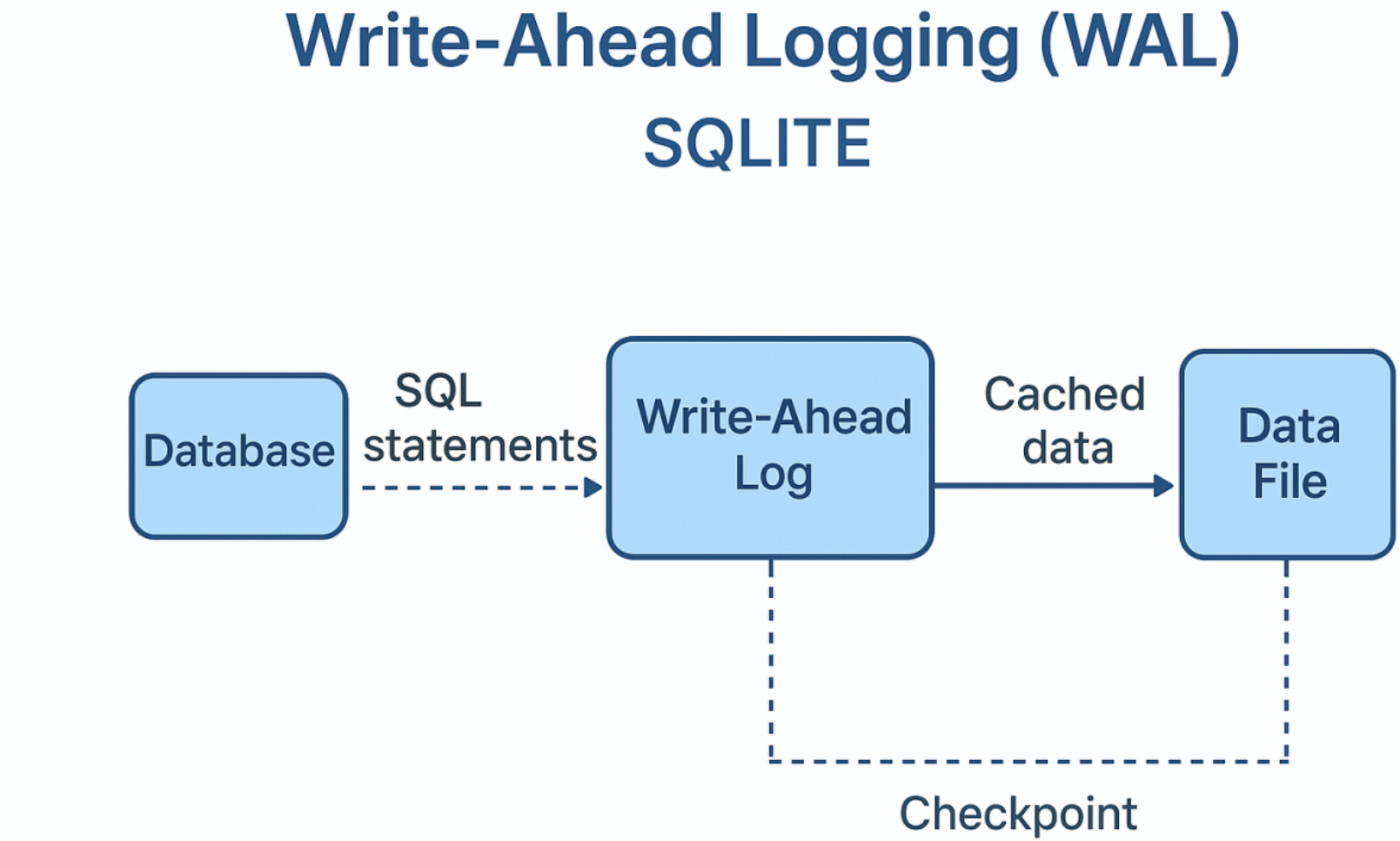


Figure 21: Write-Ahead Logging (WAL)

Source: Allan S.

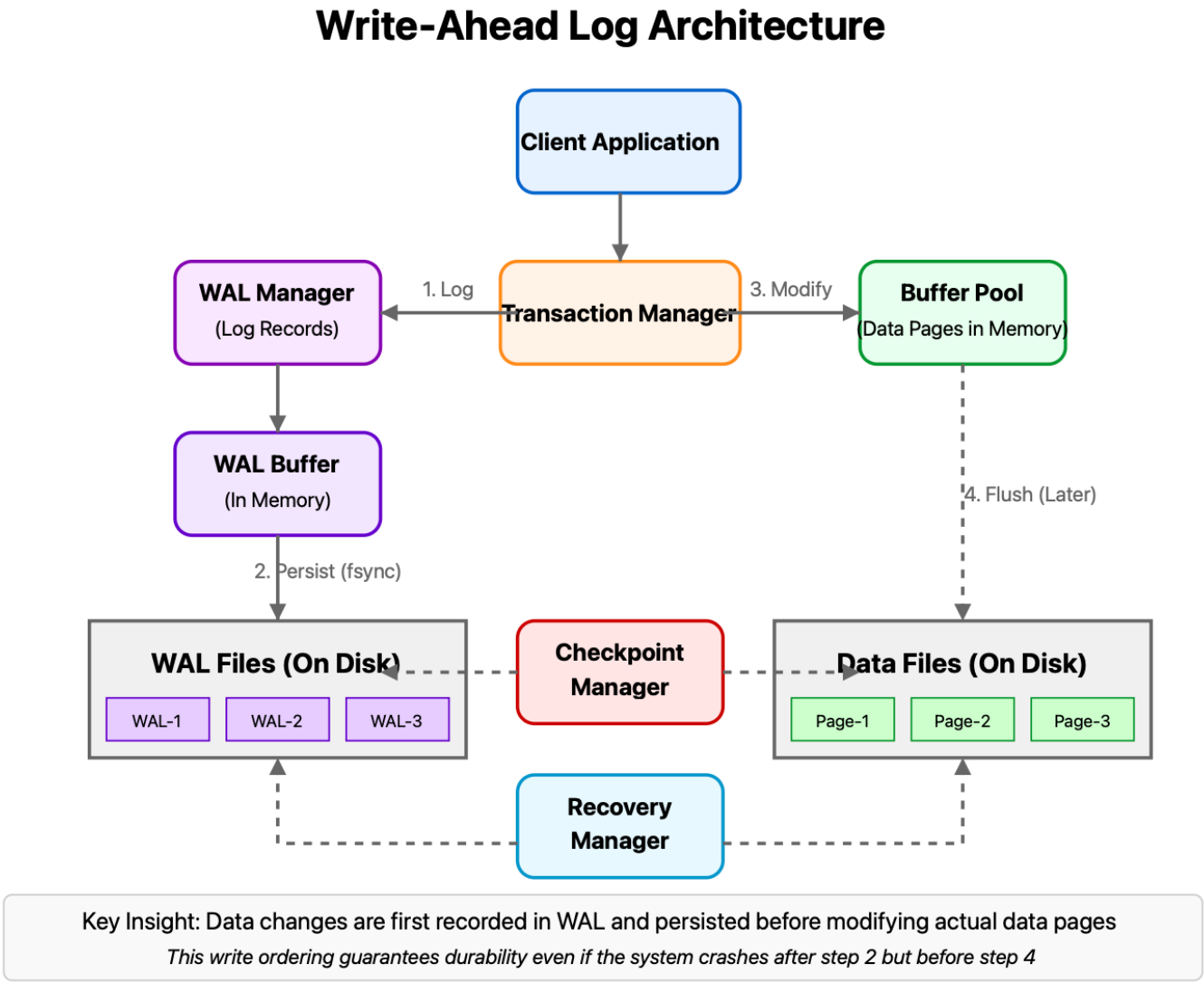


Figure 22: Write-Ahead Logs

Source: System Design & Architecture

1.8 Popular Database Systems

1.8.1 DBMS (Database Management System):

- Software used to **create, store, manage, and access data** in a database.
- Provides functions such as **CRUD, transactions, security, concurrency control, and recovery**.
- Can support different database models, such as relational, document, key-value, and graph databases.
- Examples: **MySQL, PostgreSQL, MongoDB, and Redis**.

1.8.2 DBMS vs RDBMS:

In Short

- **DBMS:** Software that manages databases, including storing, retrieving, updating, and protecting data.
- **RDBMS:** A type of DBMS that stores data in related **tables** and typically uses **SQL**.
- **Relationship:** RDBMS is a **subset of DBMS**; every RDBMS is a DBMS, but not every DBMS is an RDBMS.

1.8.3 Popular Database Systems:

- **Relational Databases:** MySQL, PostgreSQL, Oracle Database, SQL Server

- **Document Databases:** MongoDB
- **Key-Value Databases:** Redis
- **Vector Databases:** Milvus, Pinecone, Weaviate

While all relational databases support SQL and the ACID properties, they differ in their architectures, target use cases, and licensing.

1.8.4 Oracle Database

Type: RDBMS.
License: Proprietary, Commercial.
Use Case: Enterprise, banking, government.
Key Features: High scalability, reliability, security, and advanced transaction processing.

1.8.5 MySQL / MariaDB

Type: RDBMS.

License: MySQL: GPLv2 / Commercial; MariaDB: GPLv2.

Use Case: Web applications, CMS, and online services.

Key Features: Easy to use, widely supported, and optimized for transactional web applications.

1.8.6 PostgreSQL

Type: RDBMS.

License: Open Source, PostgreSQL License.

Use Case: Enterprise, analytics, scientific, and geospatial applications.

Key Features: Advanced SQL, extensibility, JSON/JSONB, and complex queries.

1.8.7 Microsoft SQL Server

Type: RDBMS.

License: Proprietary, Commercial; Developer and Express editions available.

Use Case: Enterprise and Microsoft-based applications.

Key Features: Strong Microsoft integration, high availability, security, and T-SQL.

1.8.8 SQLite

Type: Embedded, Serverless RDBMS.

License: Public Domain.

Use Case: Mobile, desktop, browser, and IoT applications.

Key Features: Lightweight, zero-configuration, and stores the database in a single file.

1.8.9 MongoDB

Type: Document Database (NoSQL).

License: SSPL; Commercial licenses available.

Use Case: Web applications, real-time systems, and flexible data models.

Key Features: Flexible documents, nested data, horizontal scaling, and high availability.

1.8.10 Vector Databases

Type: Specialized Database for Vector Search.

License: Depends on the system; Open Source or Commercial.

Use Case: AI, LLMs, semantic search, and RAG.

Key Features: Stores embeddings and performs efficient similarity search.

- **Dedicated:** Pinecone, Milvus, Weaviate.
- **Integrated:** PostgreSQL (`pgvector`), Oracle AI Vector Search.

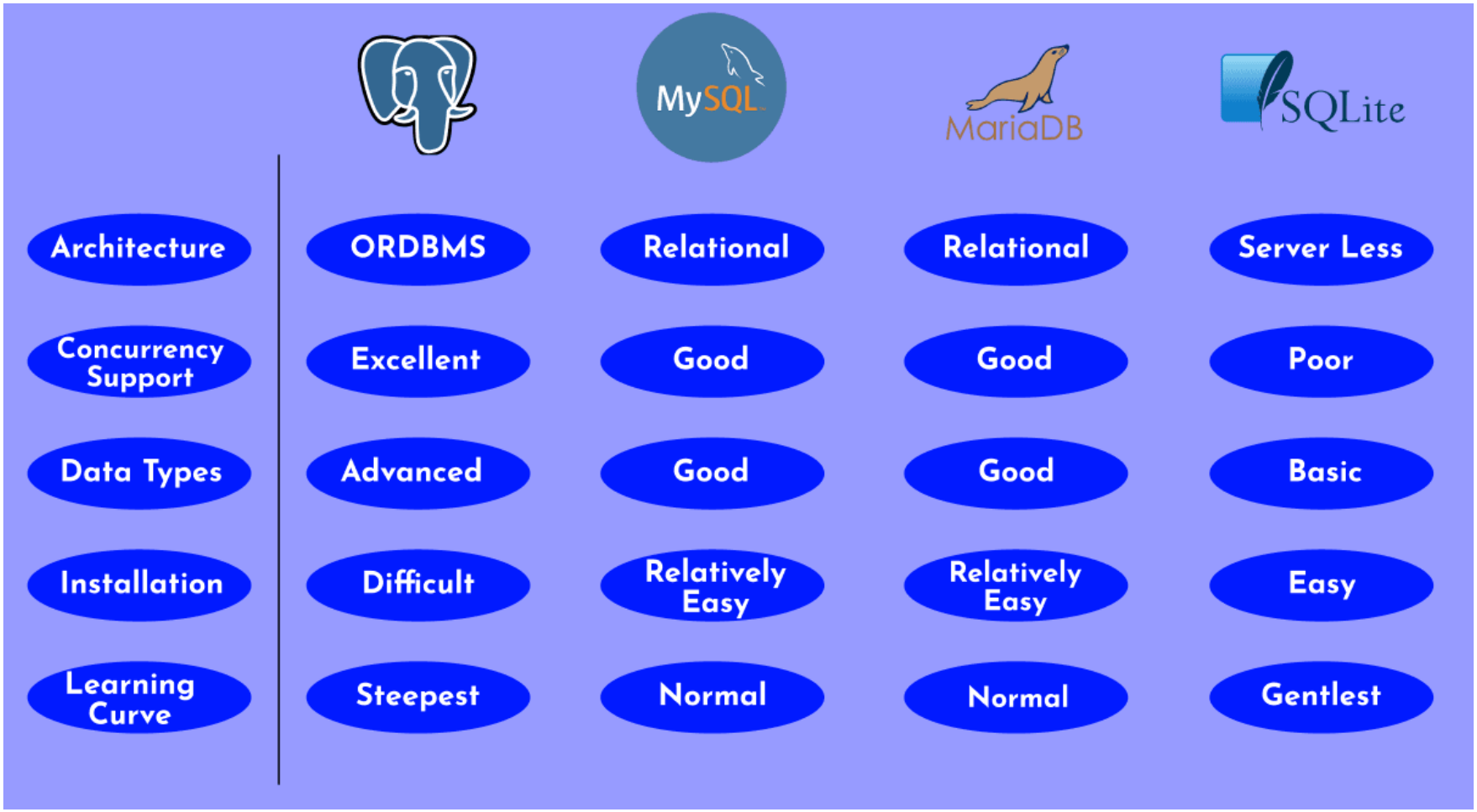


Figure 23: Comparison Matrix: PostgreSQL vs. MySQL vs. MariaDB vs. SQLite. Source: Strapi

1.8.11 ACID Support Status

Database System	RDBMS	ACID	Implementation Notes
Oracle Database	Yes	Yes	Uses Undo Segments & Redo Logs. [Docs]
MySQL	Yes	Yes	Default engine (InnoDB). *Note:* MyISAM is **not** ACID. [Docs]
MariaDB	Yes	Yes	Fork of MySQL. Uses InnoDB by default. [Docs]
PostgreSQL	Yes	Yes	Strict ACID via MVCC & WAL. [Docs]
MS SQL Server	Yes	Yes	Locking (2PL) default; Snapshot available. [Docs]
SQLite	Yes	Yes	Serialized transactions (Locking or WAL). [Docs]
MongoDB (v4.0+)	No	Yes*	Multi-document ACID transactions. [Docs]

Table 3: ACID Compliance in Modern Databases (As of 2025). Source: Official Vendor Documentation.

1.9 Chapter Summary

In this chapter, we introduced the core concepts of **Database Management Systems (DBMS)** and their role in modern applications.

We explored the **Three Levels of Data Abstraction** (External, Conceptual, and Internal) and how they provide **Logical and Physical Data Independence**, shielding users and applications from changes in the database structure or physical storage.

We then examined database **Transactions** through the lens of the **ACID properties**:

- **Atomicity & Durability:** We learned how databases use **Write-Ahead Logging (WAL)** and Recovery Managers to ensure that failed transactions can be cleanly rolled back, and committed transactions survive system crashes.
- **Consistency:** We discussed how database constraints and triggers enforce business rules.
- **Isolation:** We saw how databases use concurrency control mechanisms like **Locks (2PL)** and **Multi-Version Concurrency Control (MVCC)** to prevent anomalies (e.g., Lost Updates) and provide various **Isolation Levels**.

Finally, we reviewed popular database systems—from traditional **RDBMS** options like Oracle, MySQL, and PostgreSQL to newer paradigms like Document and Vector databases—and their support for ACID compliance.

Key Takeaways

- **Abstraction & Independence:** The 3-schema architecture separates how users view data, what data is stored logically, and how it is physically saved.
- **ACID Guarantee:** Atomicity (all-or-nothing), Consistency (correctness), Isolation (independence), Durability (persistence).
- **Concurrency Control:** Locking and Multi-Version Concurrency Control (MVCC) ensure transactions execute safely and correctly even when running concurrently.
- **Durability & Recovery:** WAL guarantees that committed data is never lost, even during sudden power failures.
- **DBMS Landscape:** Different databases (Relational, Document, Key-Value, Vector) are optimized for different use cases, but enterprise RDBMS universally provide strong ACID guarantees.

1.10 Activity/Homework: Banking System GUI

We have provided a skeleton GUI application for a banking system. Your task is to implement the backend logic to connect the application to a MySQL database and perform transactional operations.

1.10.1 Tools

MySQL

Functionality: A powerful, open-source Relational Database Management System (RDBMS) that stores and manages structured data using SQL.

Variances: MariaDB (a community-developed fork of MySQL, effectively a drop-in replacement).

phpMyAdmin

Functionality: A free software tool written in PHP, intended to handle the administration of MySQL over the Web. It supports operations on databases, tables, columns, relations, indexes, users, permissions, etc.

Tkinter

Functionality: The standard Python interface to the Tk GUI toolkit. It provides widgets such as buttons, labels, and text boxes to build desktop applications.

PyMySQL

Functionality: A pure-Python MySQL client library. It allows Python programs to connect to a MySQL server and execute SQL queries.

1.10.2 Database Schema

You must create the following tables in your MySQL database to support the application:

1. **Customers:** Stores user identity.
   - `customer_id` (int, PK), `name` (text), `tax_id` (text, unique).
2. **Accounts:** Stores individual bank accounts.
   - `account_id` (int, PK), `customer_id` (FK), `balance` (decimal).
3. **Transactions:** Stores an audit log of all operations.
   - `transaction_id` (int, PK), `account_id` (FK), `transaction_type` (text), `amount` (decimal), `created_at` (timestamp).
4. **BankReserves:** Stores the total cash holding of the bank branch.
   - `branch_id` (int, PK), `total_reserve` (decimal).
5. **AllCustomerTransactions (View):** A joined view for reporting.
   - Logic: Join `Customers` $\bowtie$ `Accounts` $\bowtie$ `Transactions`.
   - Columns: `CustomerName`, `AccountID`, `Type`, `Amount`, `Date`.

1.10.3 Lab Tasks

Open the provided `banking_gui.py` file and complete the following TODO sections:

1. **Server Connection:** Implement logic to `connect()` and `disconnect()` from the MySQL database server. Handle connection errors gracefully.
2. **Banking Operations:**
   (a) **Open Account:** Implement SQL to insert a new customer and account.
   (b) **Deposit (Transaction):**
      - Update the specific `Account` balance ($+Amount$).
      - Update the `BankReserves` total reserve ($+Amount$).
      - **Requirement:** These updates must be wrapped in a transaction block. If one fails, both must roll back.
   (c) **Withdraw (Transaction):**
      - Check if the `Account` has sufficient funds ($Balance \geq Amount$).
      - Update the `Account` balance ($-Amount$).
      - Update the `BankReserves` total reserve ($-Amount$).
      - **Requirement:** All steps must be atomic.
   (d) **Transfer (Transaction):** The core ACID test. Move money from Account A to Account B.
      - Wrap operations in a `BEGIN ... COMMIT` transaction block.

- If any step fails (e.g., insufficient funds), `ROLLBACK` the entire transaction.

(e) **Check Balance:** Query and display the current balance.

(f) **Bank Statement:** Query the `AllCustomerTransactions` view to display the user's transaction history.